A practical approach to Neural Networks

Alexandre Bergel
University of Chile
http://bergel.eu

Goal of today

Give insights on how to implement Neural Networks

Understanding the scope of application

Why implementing Neural Networks?

There are many *very solid and well supported libraries* to build neural networks

So, why one would want to implement a neural network?

- To understand the machinery behind these fabulous techniques
- To make particular changes in a network
- Understanding the scope of applications

These slides…

… are a support for the tutorial

… are not meant to be understandable when read offline

… are a summary of a lecture given at the University of Chile

… incremental in their content

… do not assume any theoretical knowledge (even if some slides may look scary)

For Pharo users

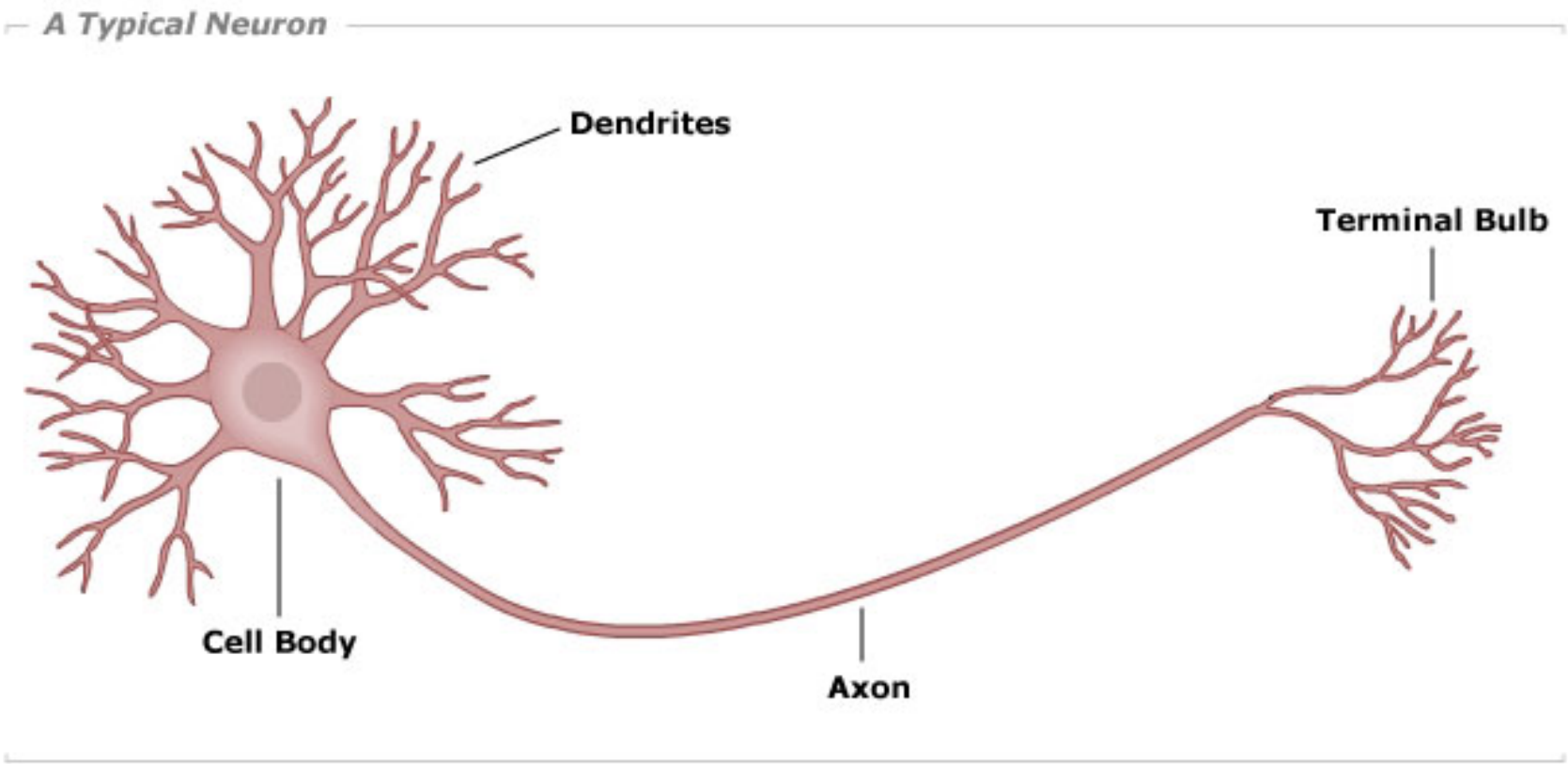
All the code presented in the tutorial is available from:

```
Gofer it
    smalltalkhubUser: 'abergel' project: 'NeuralNetworks';
    configurationOf: 'NeuralNetworks';
    loadDevelopment
```

Outline

1. Perceptron
2. Learning Perceptron
3. Sigmoid Neuron
4. Neural Network
5. Classifying some data
6. Genetic Algorithm

Perceptron



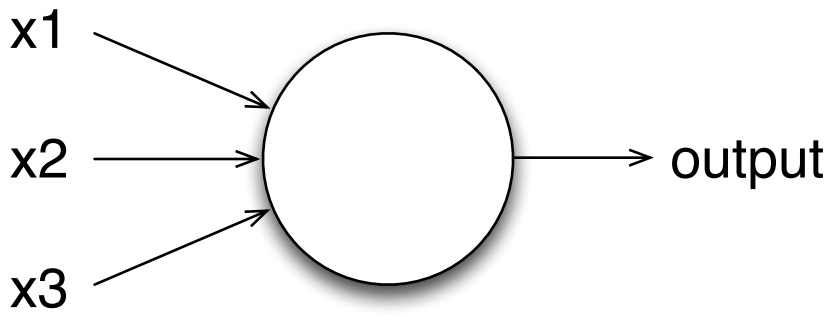
Dendrite: propagate electrochemical stimulation received from other neural cells

Axon: conducts electrical impulses away from the neuron

Perceptron

A *perceptron* is a kind of *artificial neuron*

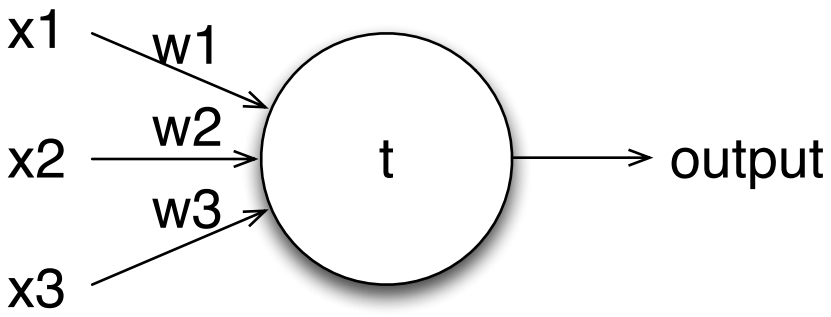
Developed in the 50s and 60s by Frank Rosenblatt, Warren McCulloch, Walter Pitts



Takes several binary inputs, x1, x2, … and produces a single binary output

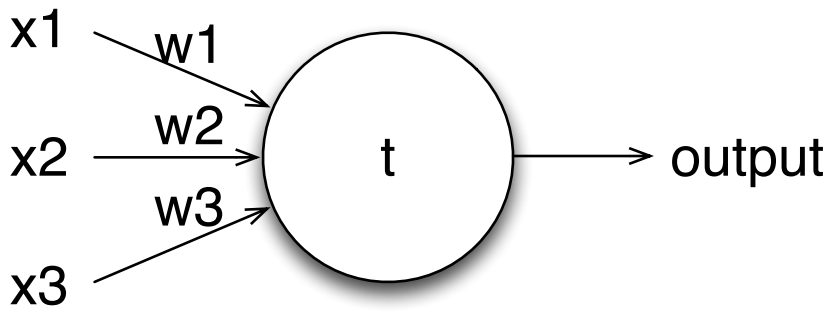
Perceptron

The output of a perceptron is the weighted sum of the input



if the weighted sum is greater than t, then we fire 1

Perceptron



$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{ threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{ threshold} \end{cases}$$

This is how a perceptron works. A perceptron is a device that makes decisions by weighing up evidence

Perceptron 

Suppose there is a great metal concert this weekend

You love metal, and you are wondering if you should go or not to the concert

You may want to make your decision by weighing up three factors:

- Is the weather good?
- Does your brother/sister/{boy,girl}friend want to accompany you?
- Is the festival near a metro stop? (You do not like driving)

Perceptron 

Suppose there is a great metal concert this weekend

You love metal, and you are wondering if you should go or not to the concert

You may want to make your decision by weighing up three factors:

x1 Is the weather good?

x2 Does your brother/sister/{boy,girl}friend want to accompany you?

x3 Is the festival near a metro stop? (You do not like driving)

Perceptron 

x1 Is the weather good?

x2 Does your brother/sister/{boy,girl}friend want to accompany you?

x3 Is the festival near a metro stop? (You do not like driving)

If you are a true-metal lover who love to share, then you may want to go even if the weather is bad and there is no stop near-by.

In that case, w1 = 2, w2 = 6, w3 = 2

Perceptron 

x1 Is the weather good?

x2 Does your brother/sister/{boy,girl}friend want to accompany you?

x3 Is the festival near a metro stop? (You do not like driving)

… or if you wish to not weak up your parents late in the evening: w1 = 1, w2 = 1, w3 = 8

Perceptron 

We are using the perceptron to model a simple decision-making.

If we pick 5 as our threshold, then we have the following condition:

$$(x1 * w1) + (x2 * w2) + (x3 * w3) >= 5$$

If the condition is true, then the perceptron outputs 1, else it output 0

Perceptron 

Varying the weights and the threshold produces a new model of decision-making

x1 Is the weather good?

x2 Does your brother/sister/{boy,girl}friend want to accompany you?

x3 Is the festival near a metro stop? (You do not like driving)

w1 = 2	w1 = 2
w2 = 6	w2 = 6
w3 = 2	w3 = 2
t = 5	t = 2

Decreasing t means that you are more willing to the metal party

Adapting the perceptron

We can move the threshold to the other side of the equation: *threshold* is now named *bias*

*(x1 * w1) + (x2 * w2) + (x3 * w3) >= 5*
*(x1 * w1) + (x2 * w2) + (x3 * w3) - 5 >= 0*

The perception can be rewritten

$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

We want some code!

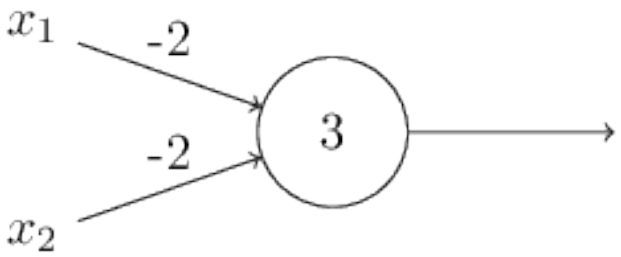
```
Object subclass: #MPPerceptron
    instanceVariableNames: 'weights bias'
    classVariableNames: ''
    package: 'NeuralNetworks-Core'
```

```
MPPerceptron>>feed: inputs
    | r tmp |
    tmp := inputs with: weights collect: [ :x :w | x * w ].
    r := tmp sum + bias.
    output := r > 0 ifTrue: [ 1 ] ifFalse: [ 0 ].
    ^ output
```

Formulating logical equation

AND, OR, NAND are elementary logical function

Consider the following perceptron:



With 00 (shortcut for x1 = 0, x2 = 0) produces the output 1, since (-2)*0 + (-2)*0 + 3 = 3, which is positive

The input 01 and 10 produces 1. But 11 produces -1, which is negative. Our perceptron implements a NAND gate.

$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

Learning Perceptron

Perceptron learning algorithm (1/2)

During the training period, a series of inputs are presented to the perceptron

Each input follows x = (x1, x2, …, xN)

For each input set, there is a *desired output (0 or 1)*

The actual output is determined by w*x + b

If the actual output is wrong, then some learning has to happen

Perceptron learning algorithm (2/2)

```
diff = desiredOutput - realOutput

lr = 0.1

weightN = oldWeightN + (lr * inputN * diff)

lr is called the learning rate
```

```
Perceptron>>train: inputs desiredOutput: desiredOutput
    | realOutput difference oldWeight currentInput learningRate newWeight |
    realOutput := self feed: inputs.
    difference := desiredOutput - realOutput.
    learningRate := 0.1.
    1 to: weights size do: [ :index |
        oldWeight := weights at: index.
        currentInput := inputs at: index.
        newWeight := oldWeight + (learningRate * currentInput * difference).
        weights at: index put: newWeight.

        bias := bias + (learningRate * difference). ]
```

```
testBasicLearning

    | p |
    p := MPPerceptron new.
    p weights: { -1 . -1 }.
    p bias: 0.5.

    100 timesRepeat: [
        p train: { 0 . 0} desiredOutput: 0.
        p train: { 0 . 1} desiredOutput: 1.
        p train: { 1 . 0} desiredOutput: 1.
        p train: { 1 . 1} desiredOutput: 1.
        ].

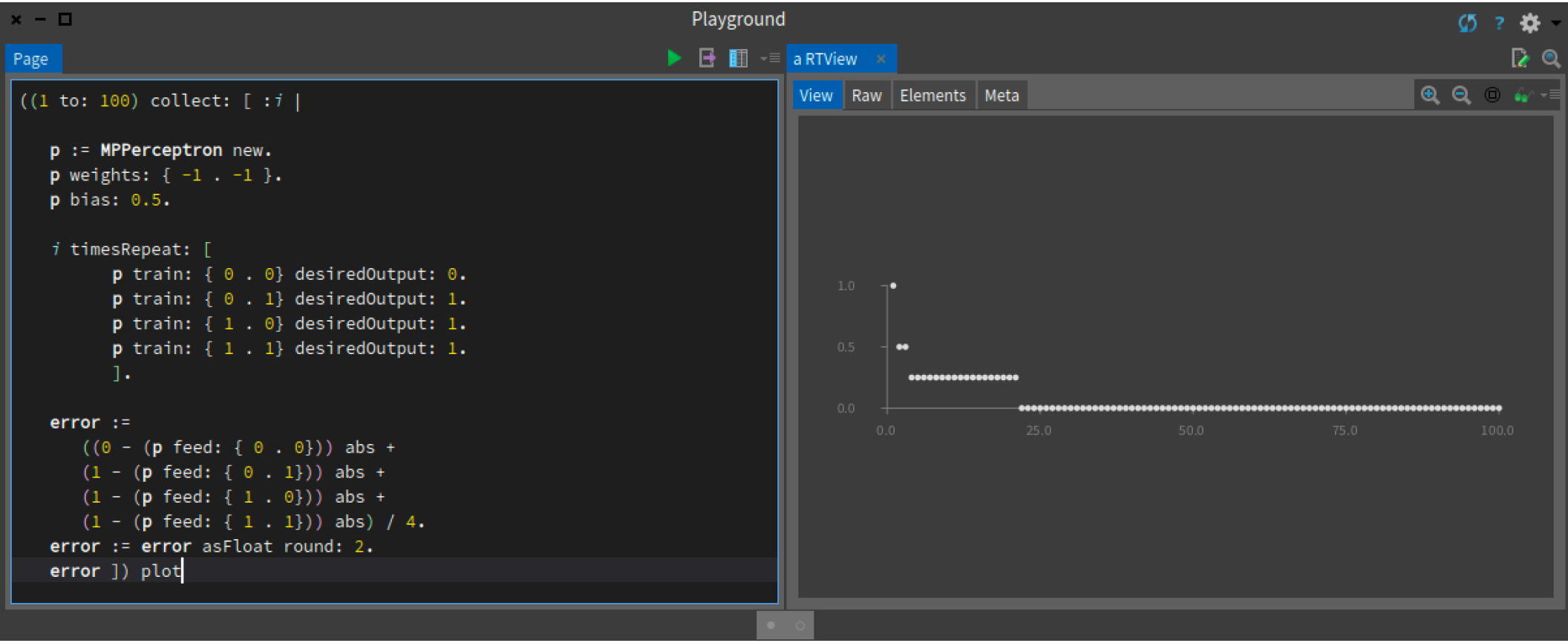
    self assert: (p feed: { 0 . 0}) equals: 0.
    self assert: (p feed: { 0 . 1}) equals: 1.
    self assert: (p feed: { 1 . 0}) equals: 1.
    self assert: (p feed: { 1 . 1}) equals: 1.
```

```
(1 to: 100) collect: [ :i |

    p := MPPerceptron new.
    p weights: { -1 . -1 }.
    p bias: 0.5.

    i timesRepeat: [
            p train: { 0 . 0} desiredOutput: 0.
            p train: { 0 . 1} desiredOutput: 1.
            p train: { 1 . 0} desiredOutput: 1.
            p train: { 1 . 1} desiredOutput: 1.
            ].

    error :=
        ((0 - (p feed: { 0 . 0})) abs +
        (1 - (p feed: { 0 . 1})) abs +
        (1 - (p feed: { 1 . 0})) abs +
        (1 - (p feed: { 1 . 1})) abs) / 4.
    error := error asFloat round: 2.
    error ]
```


Playground
Page
((1 to: 100) collect: [:i |
p := MPPerceptron new.
p weights: { -1 . -1 }.
p bias: 0.5.
i timesRepeat: [
p train: { 0 . 0} desiredOutput: 0.
p train: { 0 . 1} desiredOutput: 1.
p train: { 1 . 0} desiredOutput: 1.
p train: { 1 . 1} desiredOutput: 1.
].
error :=
((0 - (p feed: { 0 . 0})) abs +
(1 - (p feed: { 0 . 1})) abs +
(1 - (p feed: { 1 . 0})) abs +
(1 - (p feed: { 1 . 1})) abs) / 4.
error := error asFloat round: 2.
error]) plot
a RTView
View
Raw
Elements
Meta
1.0
0.5
0.0
0.0
25.0
50.0
75.0
100.0

Perceptron learning algorithm

DEMO

2D Points (example2dPlan example2dPlanLearningCurve)

Your Job

1. Implement the perceptron
2. Make the perceptron learn
3. Verify your perceptron can learn the AND, OR logical gate
4. *Optional: Display the learning curve*

Sigmoid Neuron

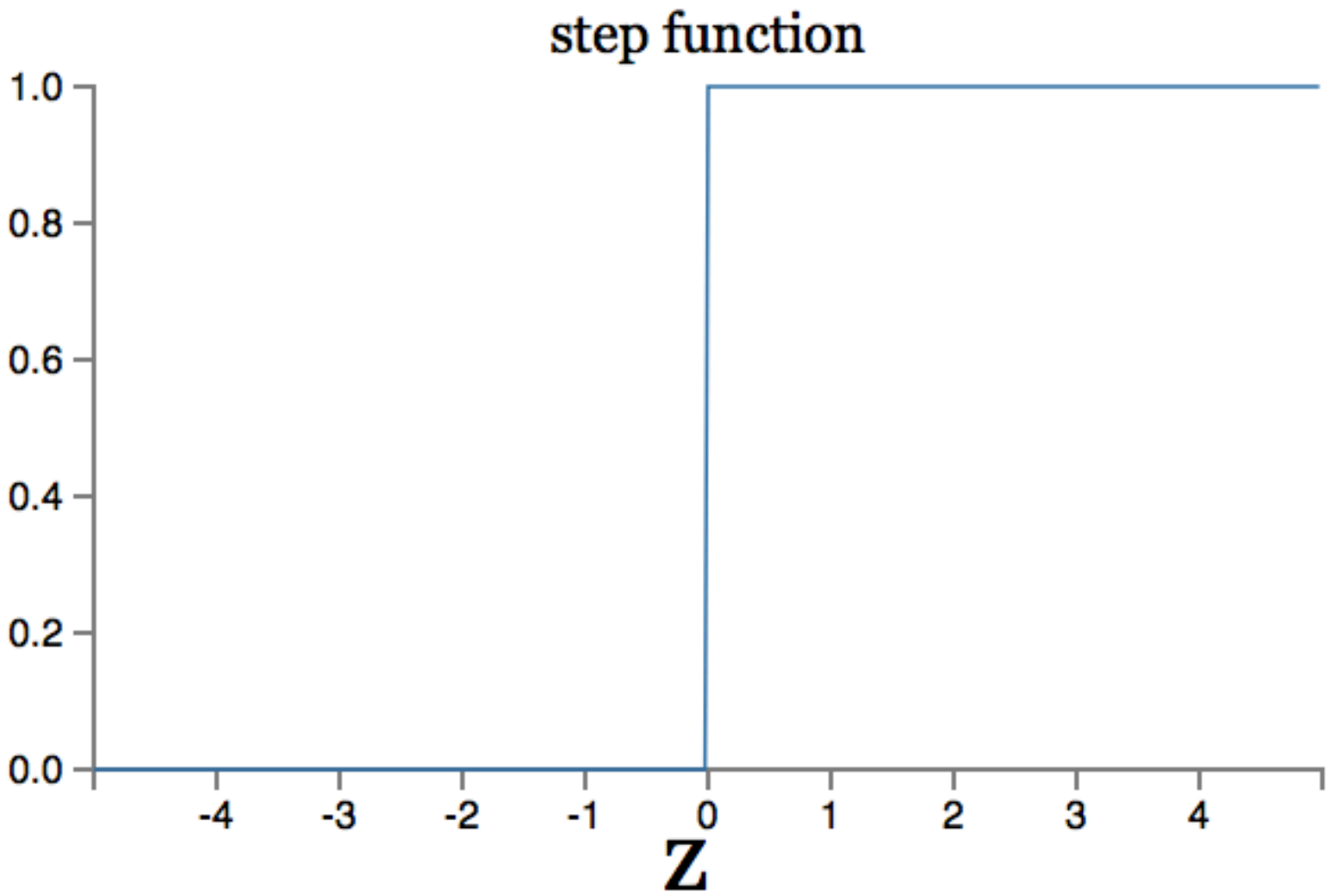
Problem with the perceptron

$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

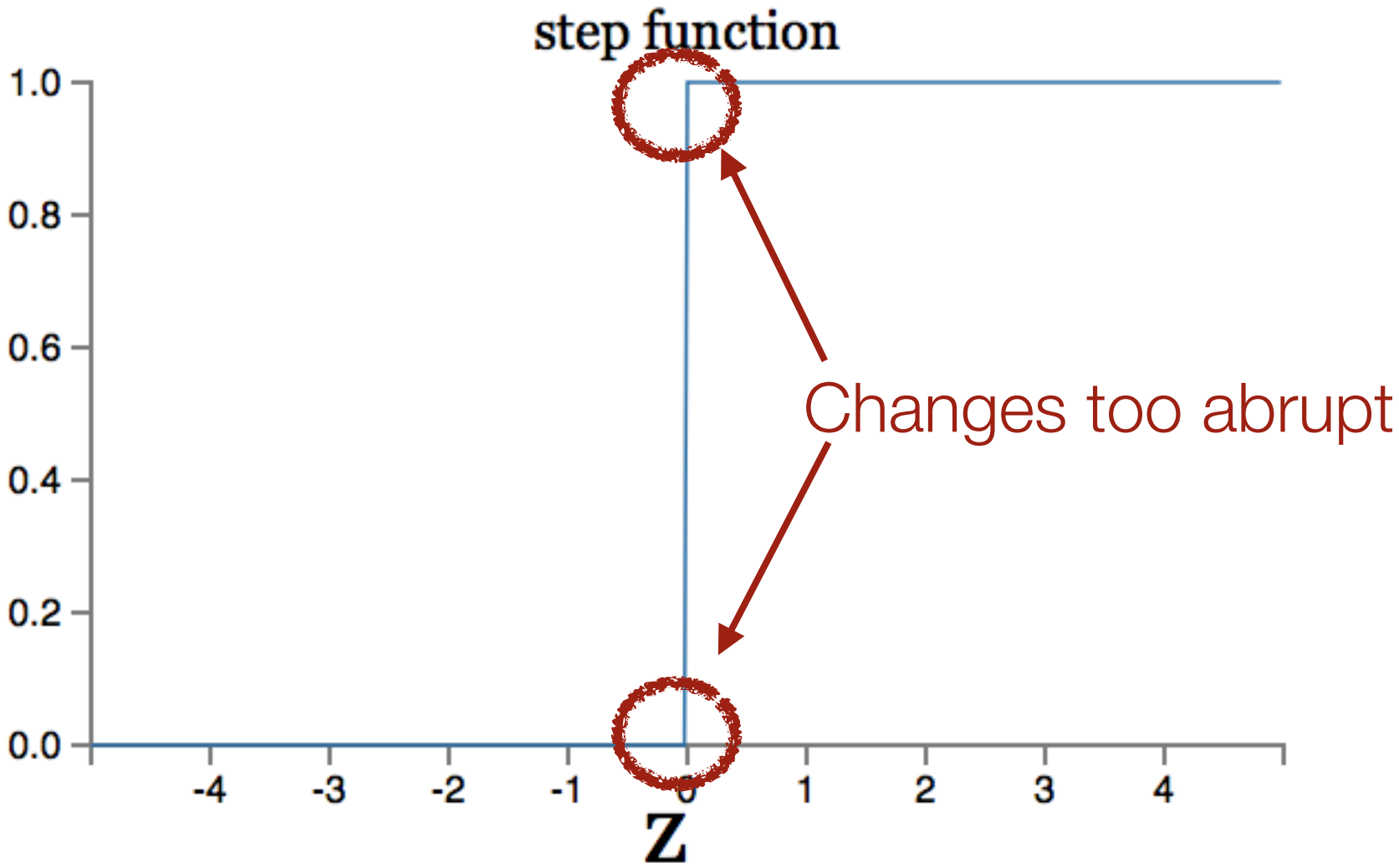
When learning from a small error, a big change can occurs in the output

=> And that is a big problem when combined with other neurons

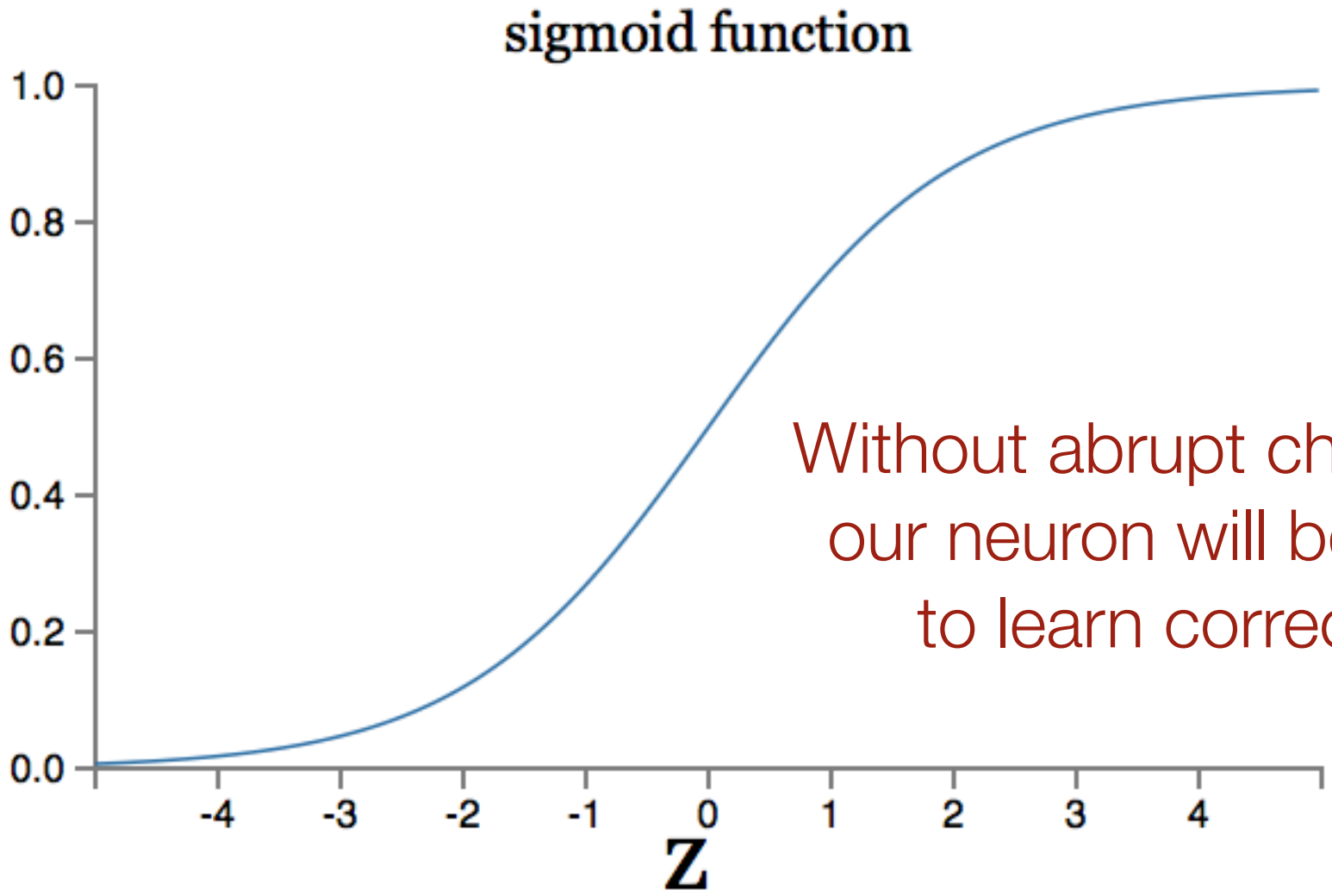
Perceptron



Perceptron


step function
1.0
0.8
0.6
0.4
0.2
0.0
-4
-3
-2
-1
0
1
2
3
4
Z
Changes too abrupt

Sigmoid neuron



Without abrupt changes, our neuron will be able to learn correctly

Sigmoid Neuron

To summarize, the output of a sigmoid neuron is

$$\frac{1}{1 + \exp(-\sum_j w_j x_j - b)}$$

Defining the sigmoid neuron

```
Object subclass: #MPNeuron
    instanceVariableNames: 'weights bias delta output'
    classVariableNames: ''
    package: 'NeuralNetworks-Core'
```

The delta and output variables will be used by the back propagation algorithm, with neural networks

```
MPNeuron>>feed: inputs
    | z |
    self assert: [ inputs isCollection ] description: 'Should be a
collection'.
    self assert: [ inputs size = weights size ] description: 'Input should
have the same size then the weights'.
    z := (inputs with: weights collect: [ :x :w | x * w ]) sum + bias.
    output := 1 / (1 + z negated exp).
    ^ output
```

```
MPNeuron>>train: inputs desiredOutput: desiredOutput
   | learningRate theError output delta |
   output := self feed: inputs.
   learningRate := 0.5.

   theError := desiredOutput - output.
   delta := (theError * (output * (1.0 - output))).

   inputs withIndexDo: [ :anInput :index |
      weights at: index put:
        ((weights at: index) + (learningRate * delta * anInput))

   bias := bias + (learningRate * delta)
```

Your Job

1. Implement the sigmoid neuron
2. Make the sigmoid neuron learn
3. Verify that a sigmoid neuron behaves as a perceptron with the AND, OR logical gates
4. *Optional: Define a class AbstractNeuron, superclass of Neuron and Perceptron*
5. *Optional: Display the learning curve*

Neural Network

Limitation of a neuron

We have seen how to train a neuron and perceptron with a NAND, OR, AND logical gate

Let's try with a XOR

```
((1 to: 500) collect: [ :i |

    p := NNNeuron new.
    p weights: { -1 . -1 }.
    p bias: 0.5.

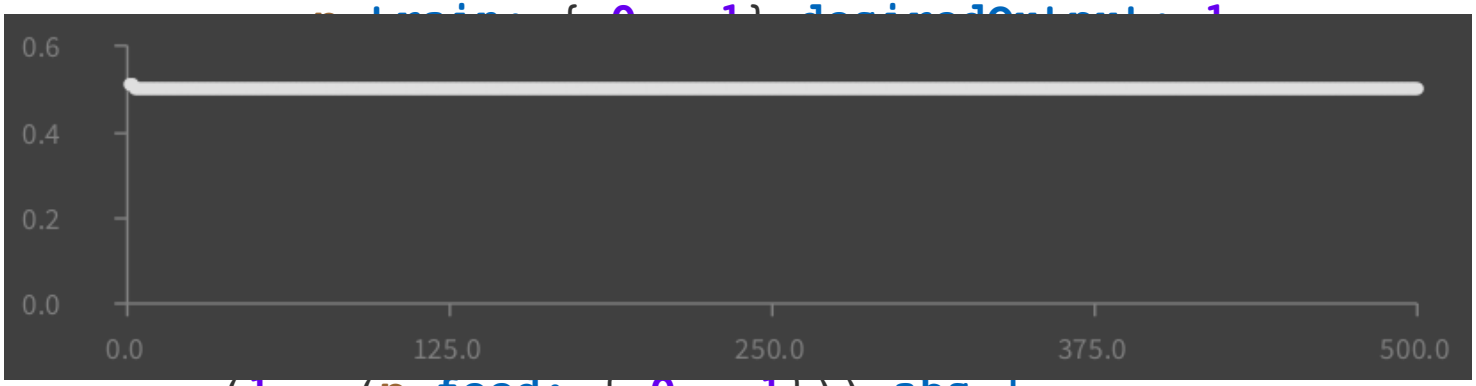
    i timesRepeat: [
            p train: { 0 . 0} desiredOutput: 0.
            p train: { 0 . 1} desiredOutput: 1.
            p train: { 1 . 0} desiredOutput: 1.
            p train: { 1 . 1} desiredOutput: 0.
            ].

    error :=
        ((0 - (p feed: { 0 . 0})) abs +
        (1 - (p feed: { 0 . 1})) abs +
        (1 - (p feed: { 1 . 0})) abs +
        (0 - (p feed: { 1 . 1})) abs) / 4.
    error := error asFloat round: 2.
    error ]) plot
```

```
((1 to: 500) collect: [ :i |

    p := MPNeuron new.
    p weights: { -1 . -1 }.
    p bias: 0.5.

    i timesRepeat: [
            p train: { 0 . 0} desiredOutput: 0.
```




```
        (1 - (p feed: { 0 . 1})) abs +
        (1 - (p feed: { 1 . 0})) abs +
        (0 - (p feed: { 1 . 1})) abs) / 4.
    error := error asFloat round: 2.
    error ]) plot
```

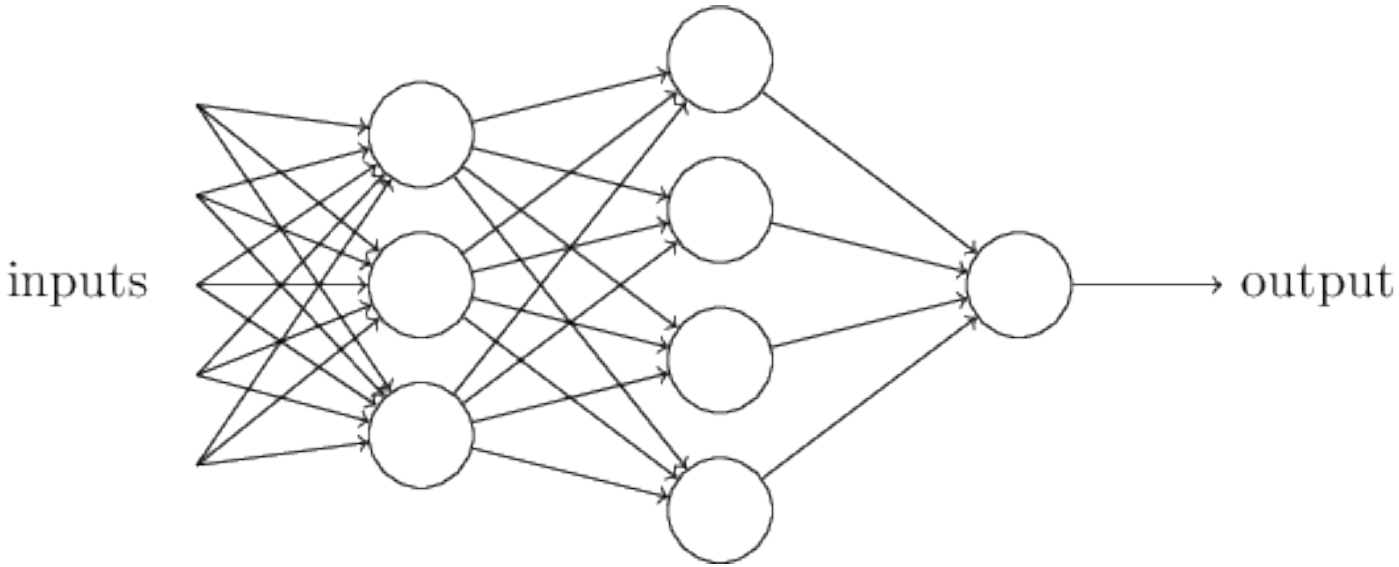
Limitation of a neuron

Our neuron cannot learn something as complex as a XOR logical gate

This is an example of why we need something more powerful than a single neuron

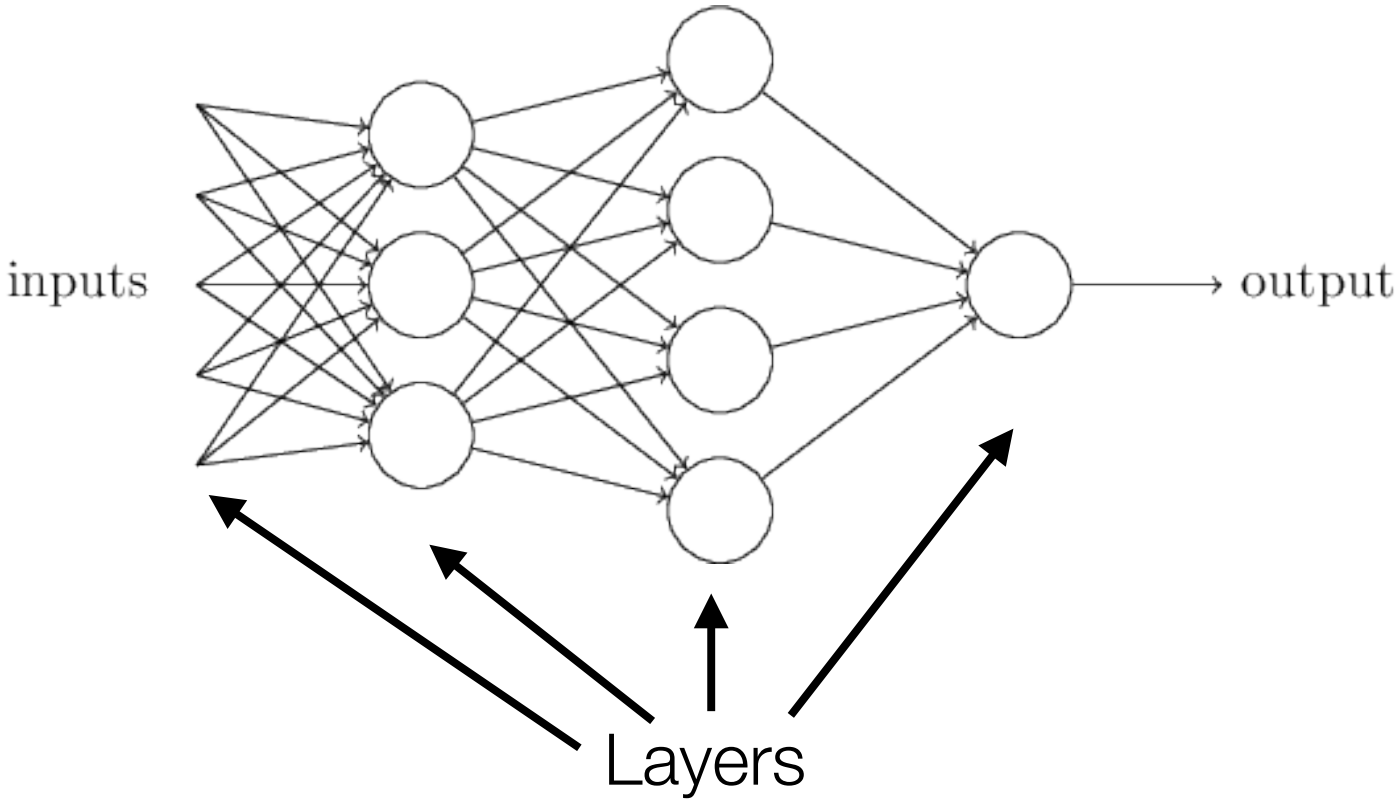
Network of neuron

A network has the following structure



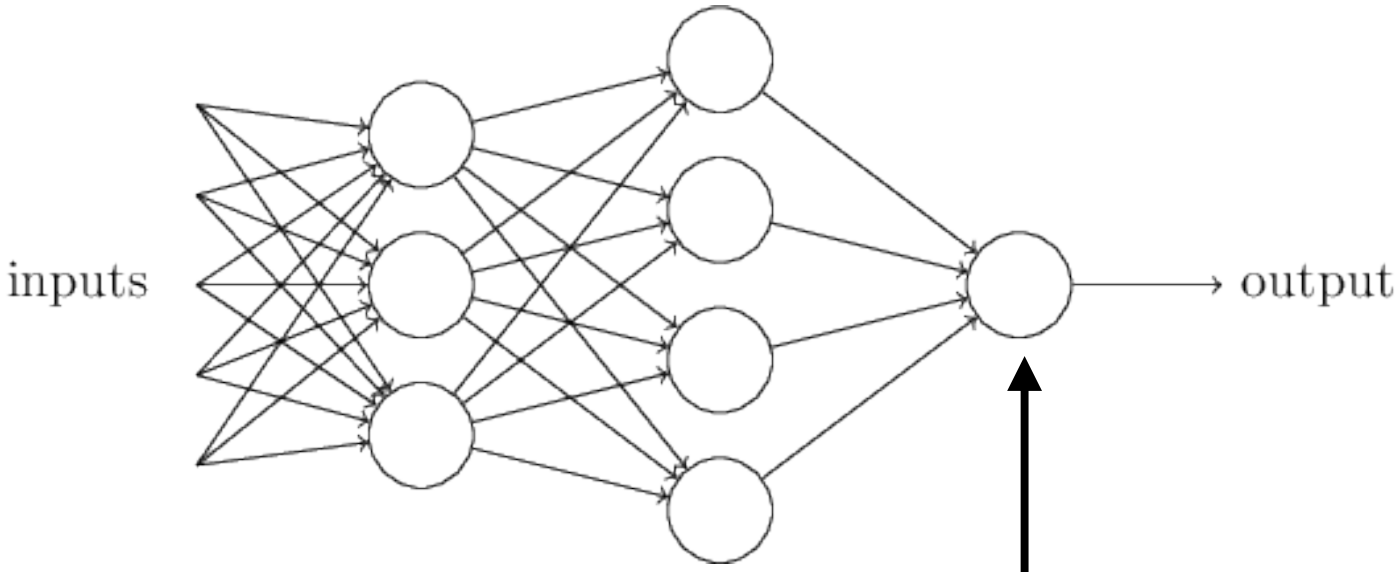
Network of neuron

A network has the following structure



Network of neuron

A network has the following structure

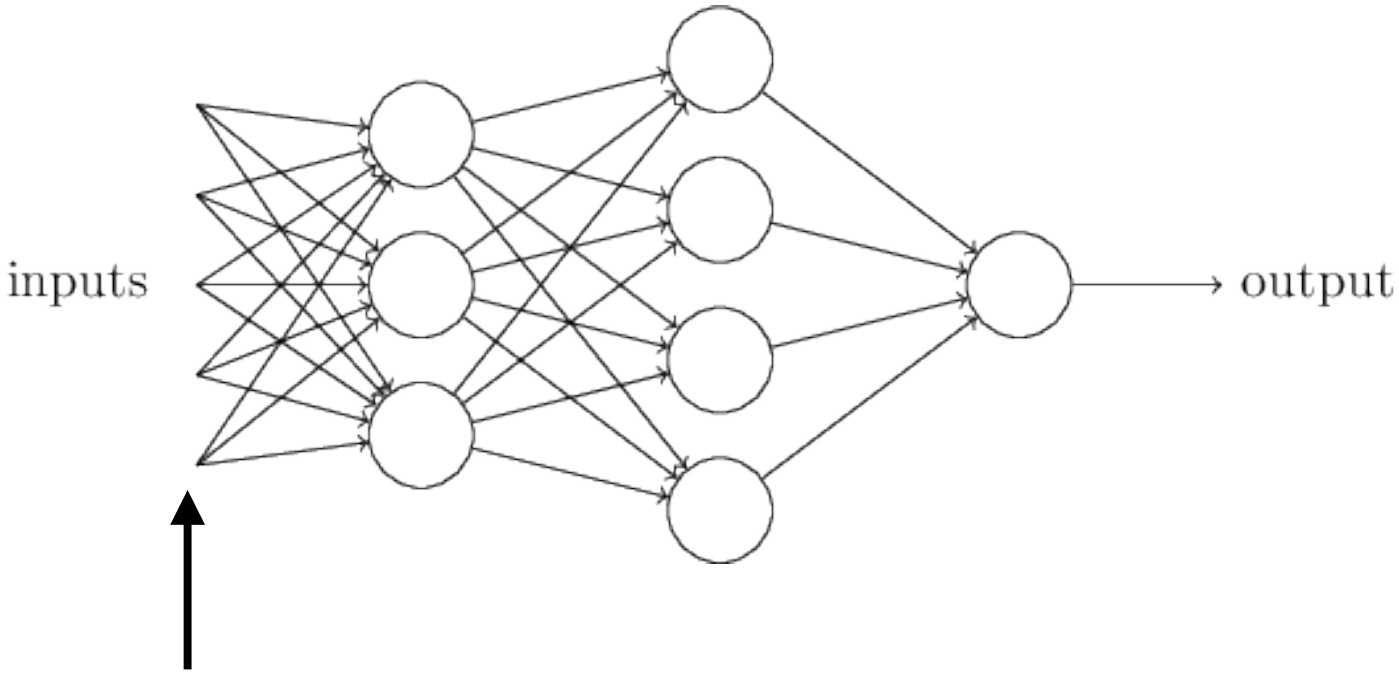


Output layer

(This example shows only one output neuron, but we could have many)

Network of neuron

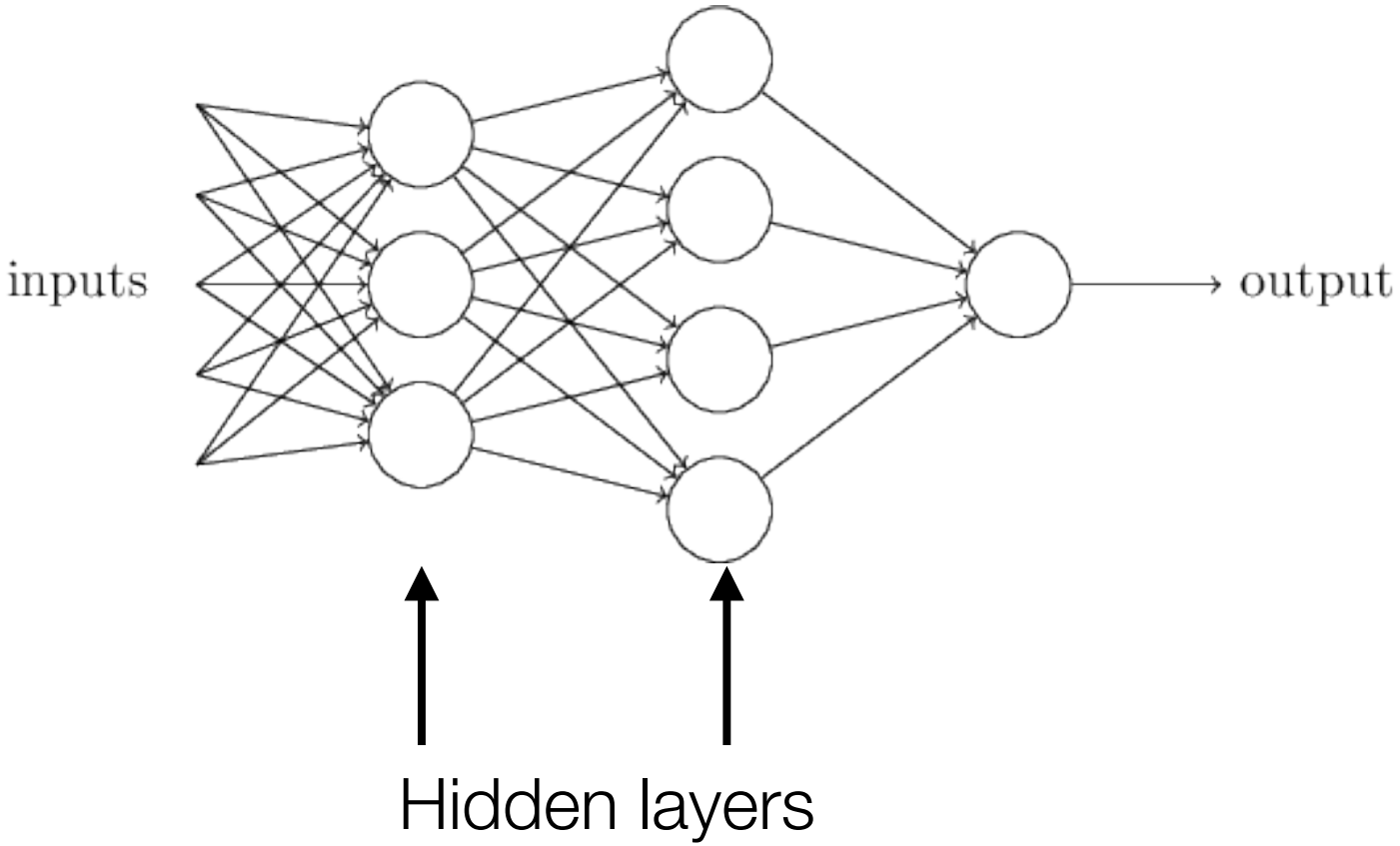
A network has the following structure



Input layer

Network of neuron

A network has the following structure



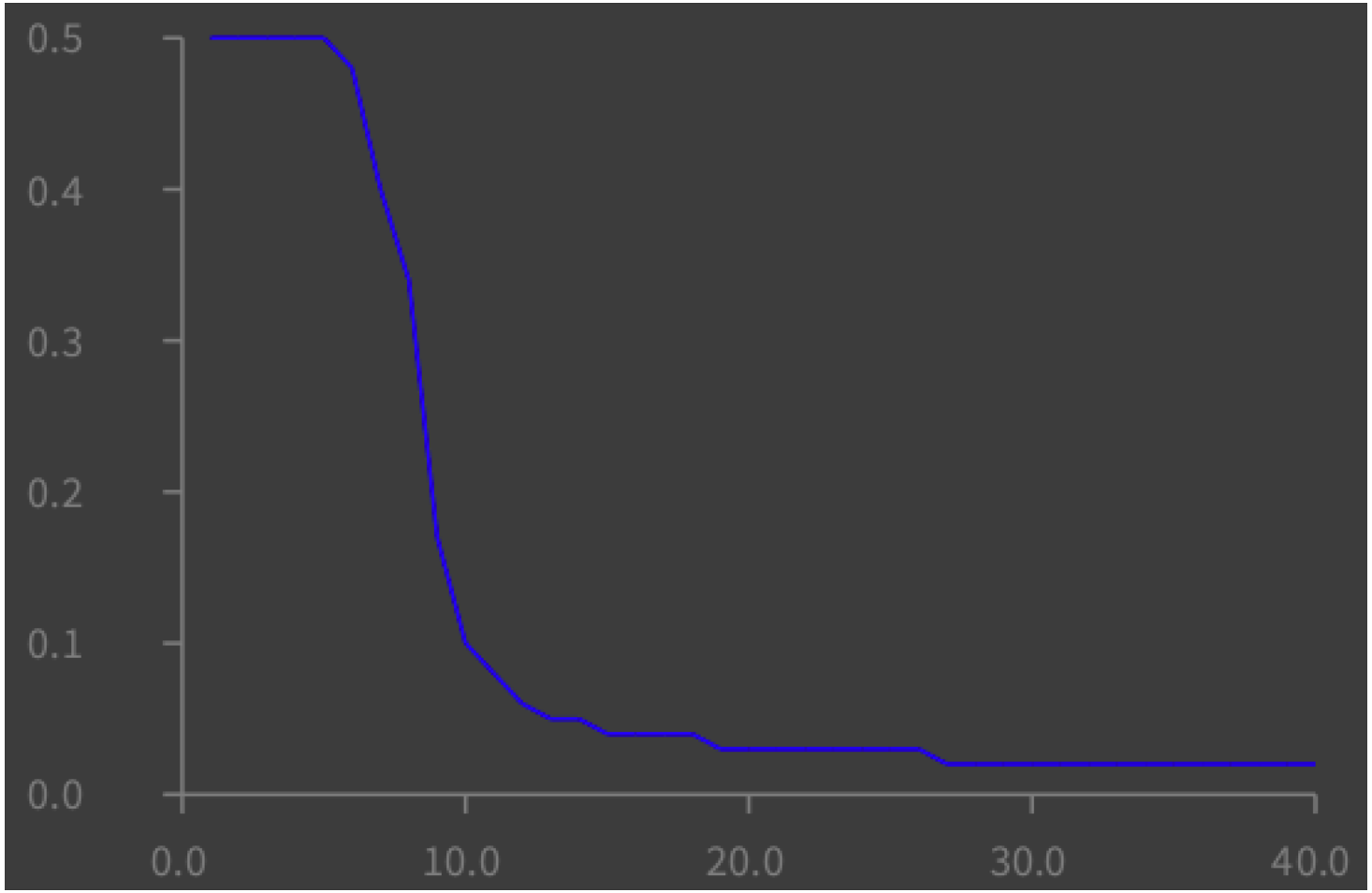
```
((1 to: 10000 by: 250) collect: [ :i |

    p := NeuralNetwork new.
    p configure: 2 numberOfHidden: 2 nbOfOutput: 1.

    i timesRepeat: [
            p train: { 0 . 0} desiredOutput: {0}.
            p train: { 0 . 1} desiredOutput: {1}.
            p train: { 1 . 0} desiredOutput: {1}.
            p train: { 1 . 1} desiredOutput: {0}.
            ].

    error :=
        ((0 - (p feed: { 0 . 0}) first) abs +
        (1 - (p feed: { 0 . 1}) first) abs +
        (1 - (p feed: { 1 . 0}) first ) abs +
        (0 - (p feed: { 1 . 1}) first) abs) / 4.
    error := error asFloat round: 2.
    error ]) plot
```

```
((1 to: 10000 by: 250) collect: [ :i |
```



Implementation support

To ease your effort, we provide an implementation, written in Pharo

The following slides assume that you have 3 classes: NeuralNetwork, NeuronLayer, and Neuron

- NeuralNetwork contains some layers
- NeuronLayer contains neurons

Feeding

Operation that consists in providing a set of inputs to the network, and obtain a set of outputs

Consider L(n) the layer at position n

Output of L(n-1) is provided to the input of L(n)

The feeding process is often called as *forward feeding*

Network & Layers

```
Object subclass: #NeuralNetwork
    instanceVariableNames: 'numberOfInputs layers learningRate random errors'
    classVariableNames: ''
    package: 'NeuralNetworks-Core'
```

```
Object subclass: #NeuronLayer
    instanceVariableNames: 'previousLayer nextLayer neurons learningRate'
    classVariableNames: ''
    package: 'NeuralNetworks-Core'
```

You need to properly initialize the network and layer.
You therefore needs some configuration methods

Feeding

```
NeuralNetwork>>feed: someInputValues

  "Feed the first layer with the provided inputs"

  ^ self firstLayer feed: someInputValues
```

Feeding

```
NeuralNetwork>>feed: someInputValues

  "Feed the first layer with the provided inputs"

  ^ self firstLayer feed: someInputValues
```



First hidden layer!

Feeding

```
NeuralLayer>>feed: someInputValues
  "Feed the neuron layer with some inputs"

  | someOutputs |
  someOutputs := neurons collect: [ :n | n feed:
someInputValues ] as: Array.
  ^ self isOutputLayer
     ifTrue: [ someOutputs ]
     ifFalse: [ self nextLayer feed: someOutputs ]
```

Making a network learning

This is a delicate step that involve a complex algorithm

Difficult to debug

The complete training process

Composed of 3 steps:

1 - forward feeding the inputs

2 - backward propagating the errores in the delta variable

3 - updating neuron weights and biases

Backpropagation

During the forward feeding, an output is produced by the network.

This output can be compared with the expected outputs

Learning implies adjustment of the network based on the difference between the actual output and the expected outputs

A new evaluation of the network goes from the output layer to the first hidden layer (direction opposite than when feedforwarding)

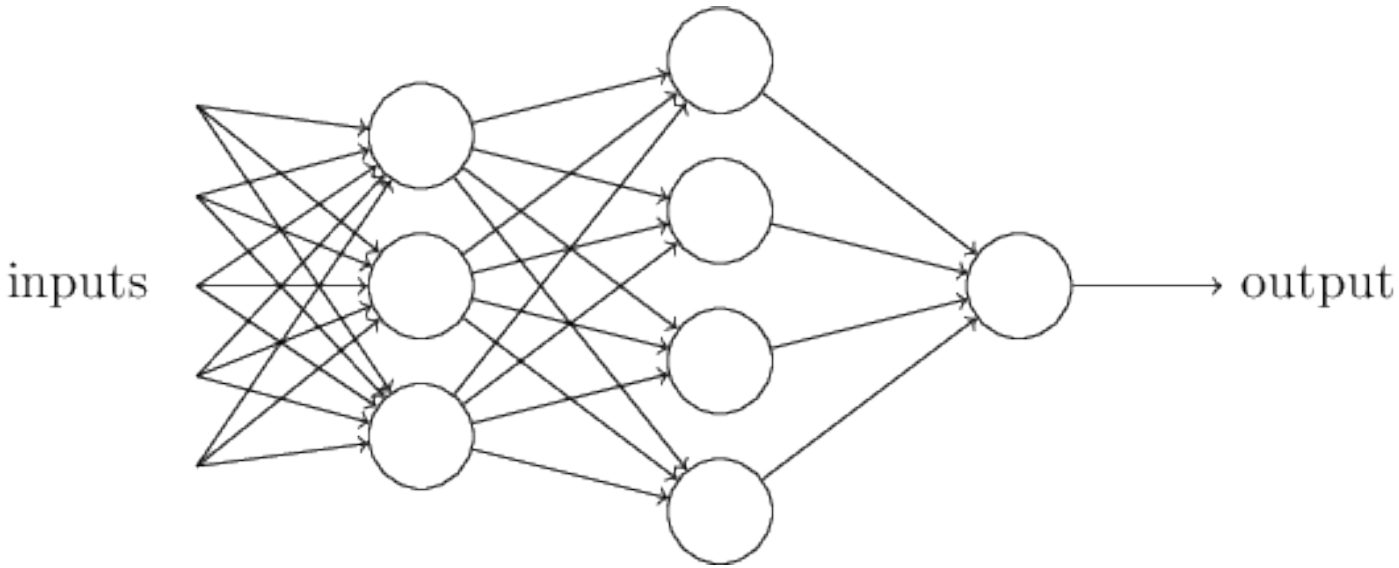
Backpropagation

Learning implies that additional information has to be kept in a neuron. In particular:

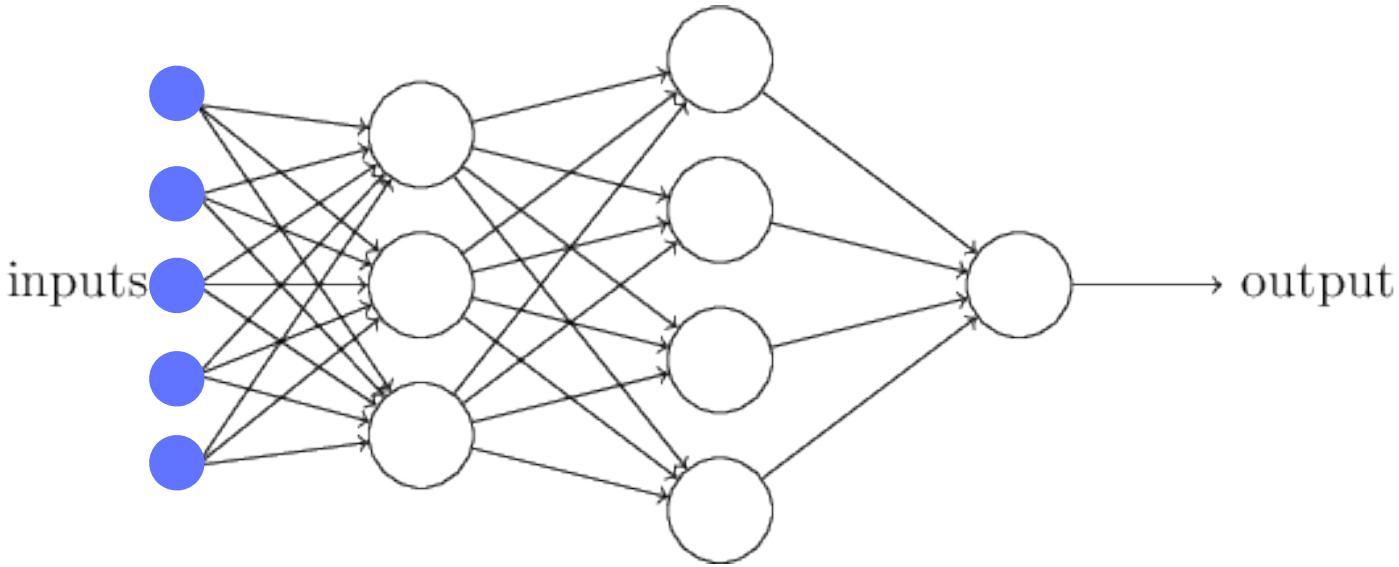
output: This variable is used to keep the value when the neuron is fed. Whenever the neuron is evaluated, keep the output value

delta: value that represents the derivative of the output multiplied with the difference *(expected output - actual output)*

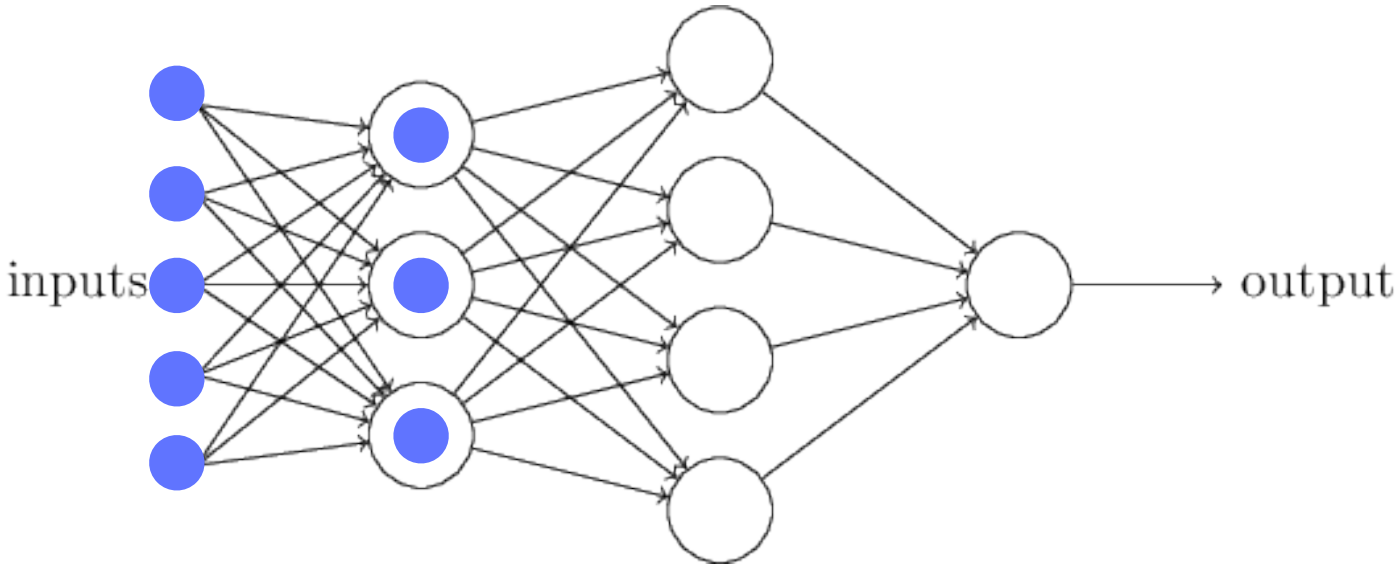
Backpropagation: Step 1 - Feeding



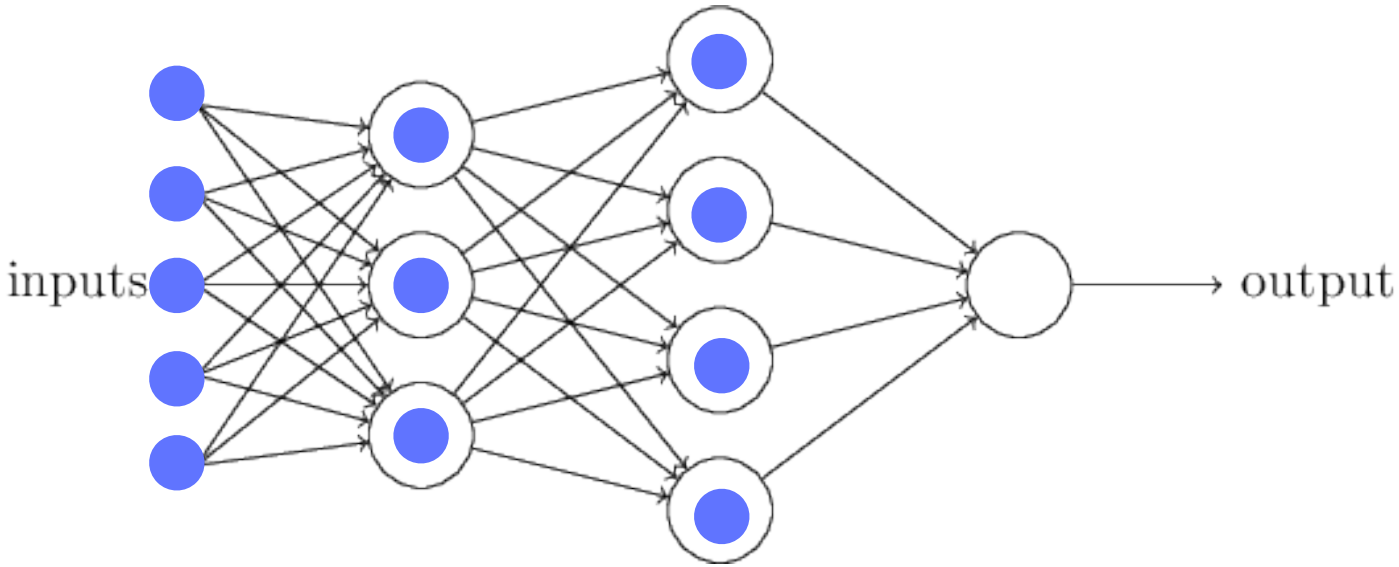
Backpropagation: Step 1 - Feeding



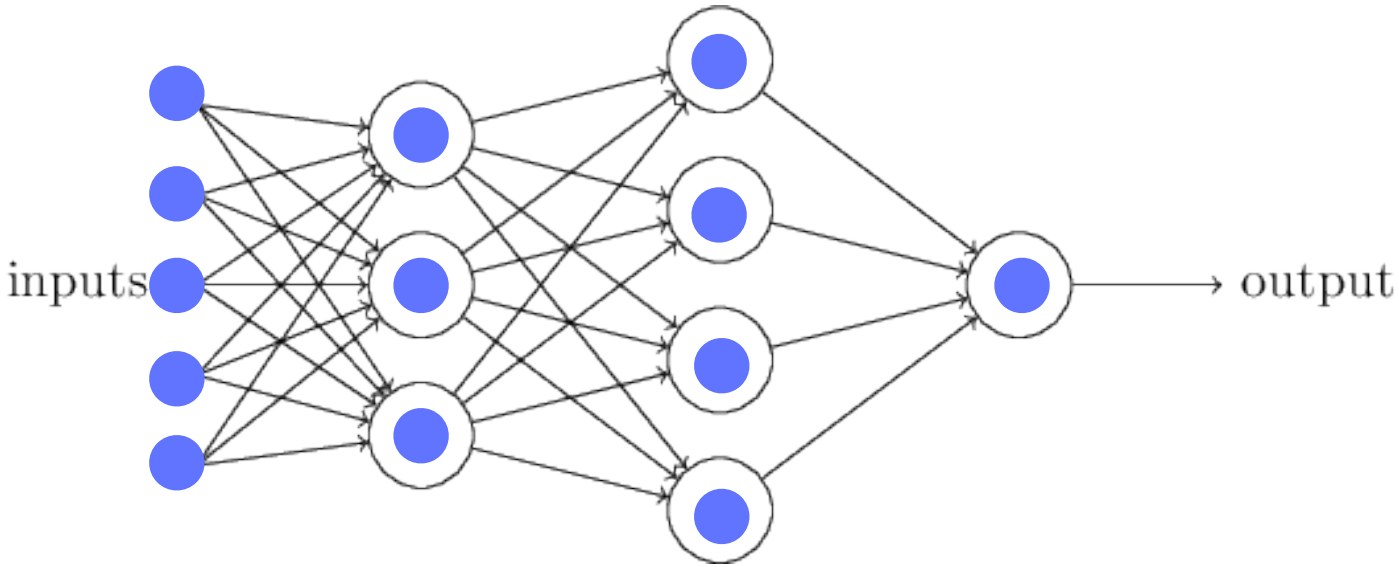
Backpropagation: Step 1 - Feeding



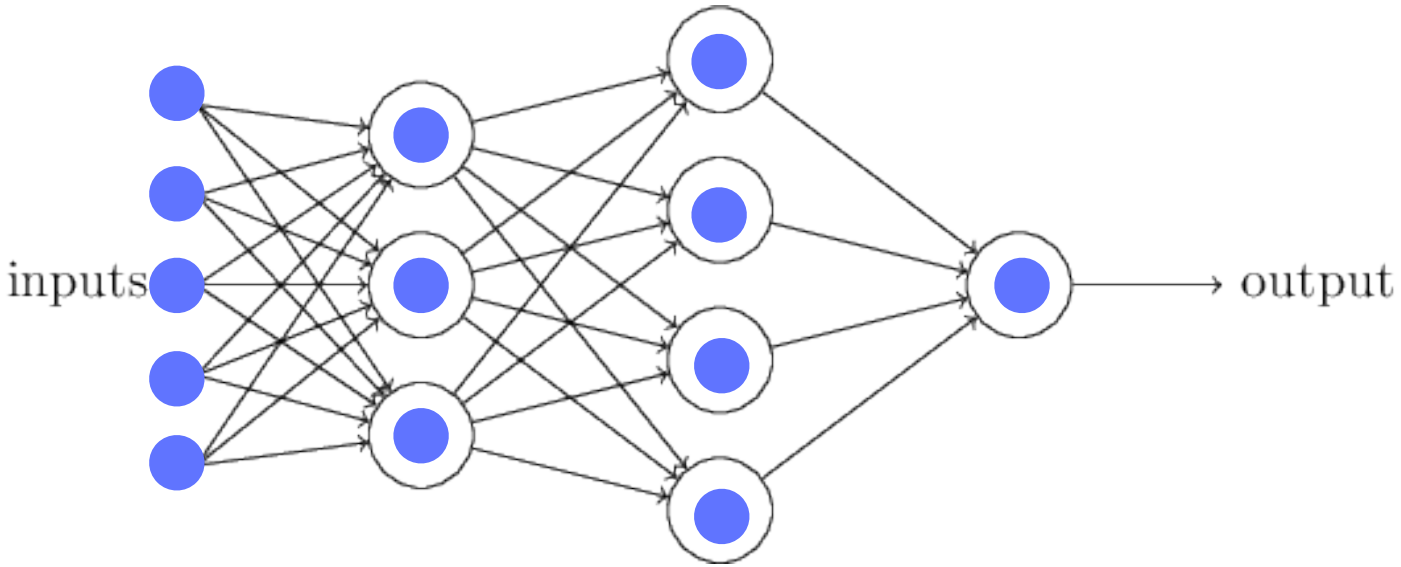
Backpropagation: Step 1 - Feeding



Backpropagation: Step 1 - Feeding

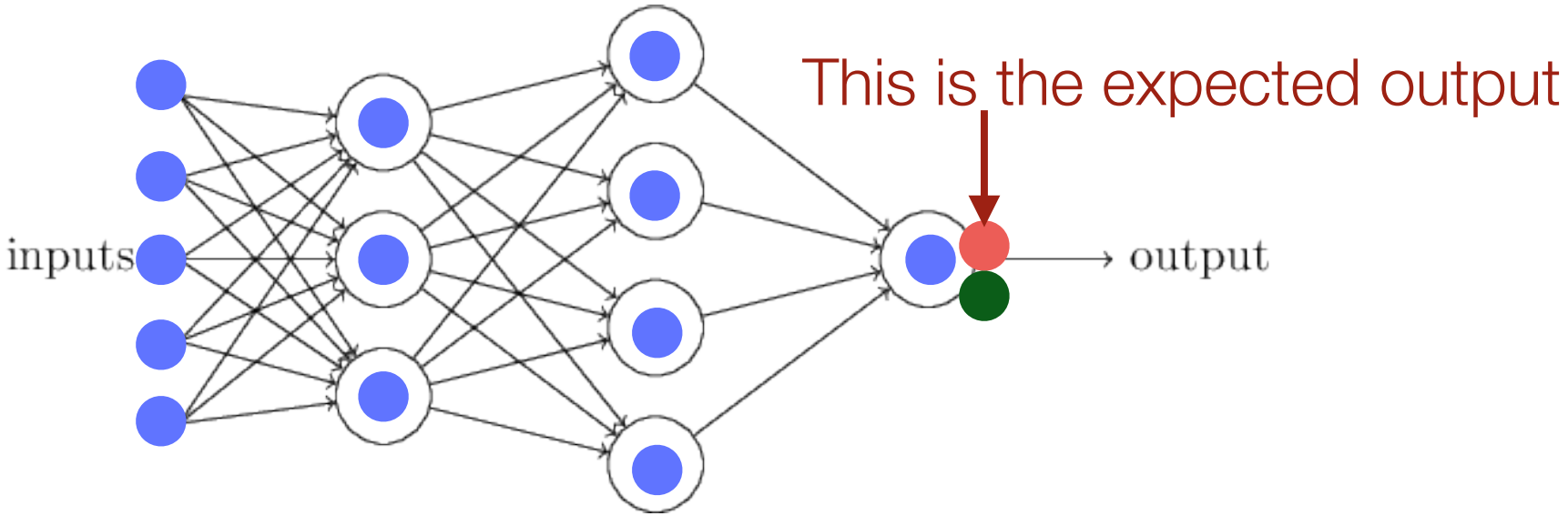


Backpropagation: Step 1 - Feeding



For each neuron,
the output value ● is kept in the neuron

Backpropagation:
Step 2 - Error backpropagation

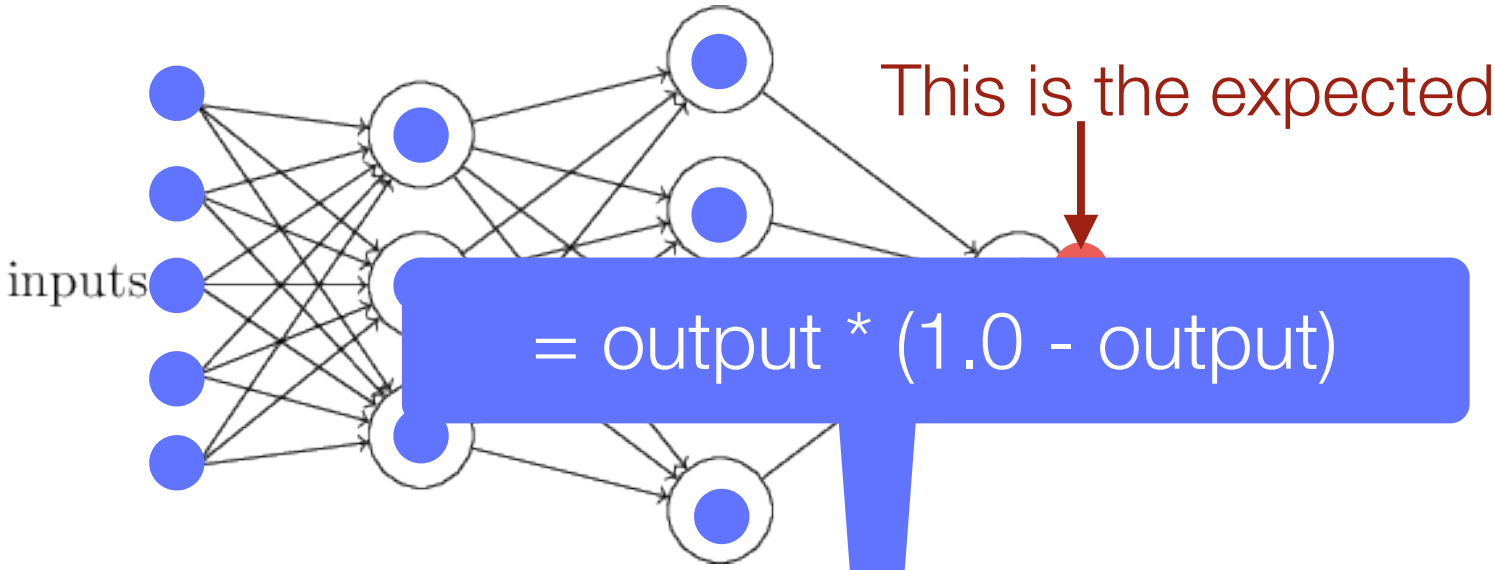


We first operate on the output layer.
The new delta of each output neuron is

error := expected output - output

delta := error * transferDerivative(output)

Backpropagation:
Step 2 - Error backpropagation

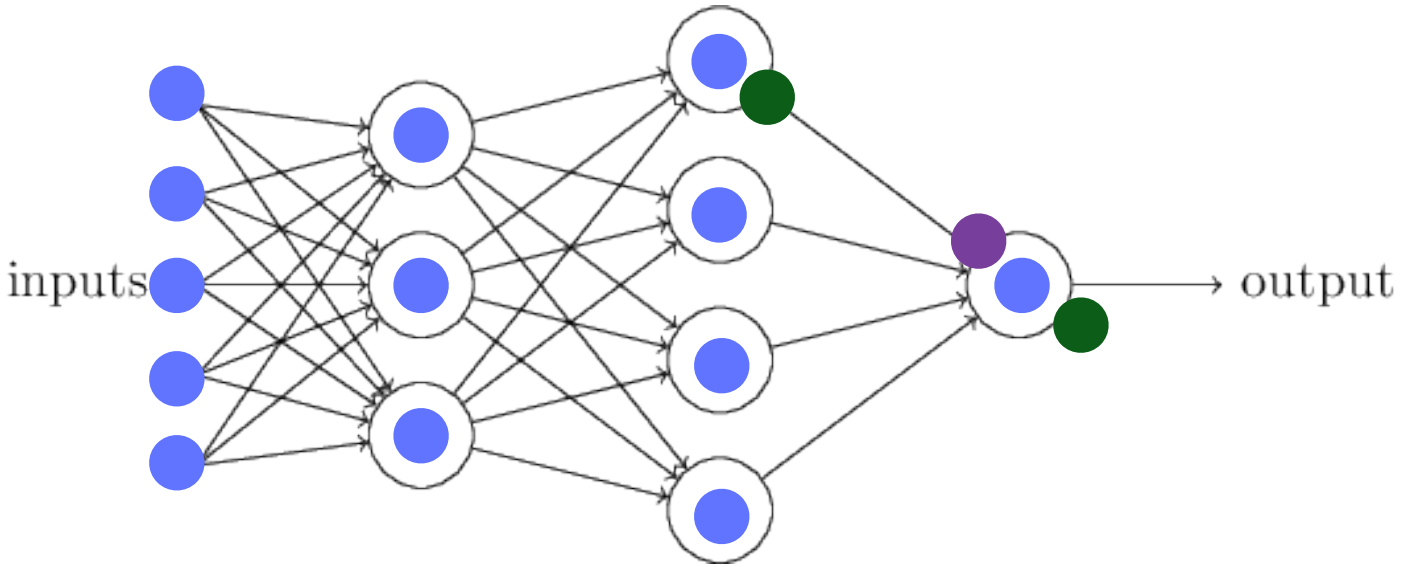


We first operate on the output layer.
The new delta of each output neuron is

error := expected output - output

delta := error * transferDerivative(output)

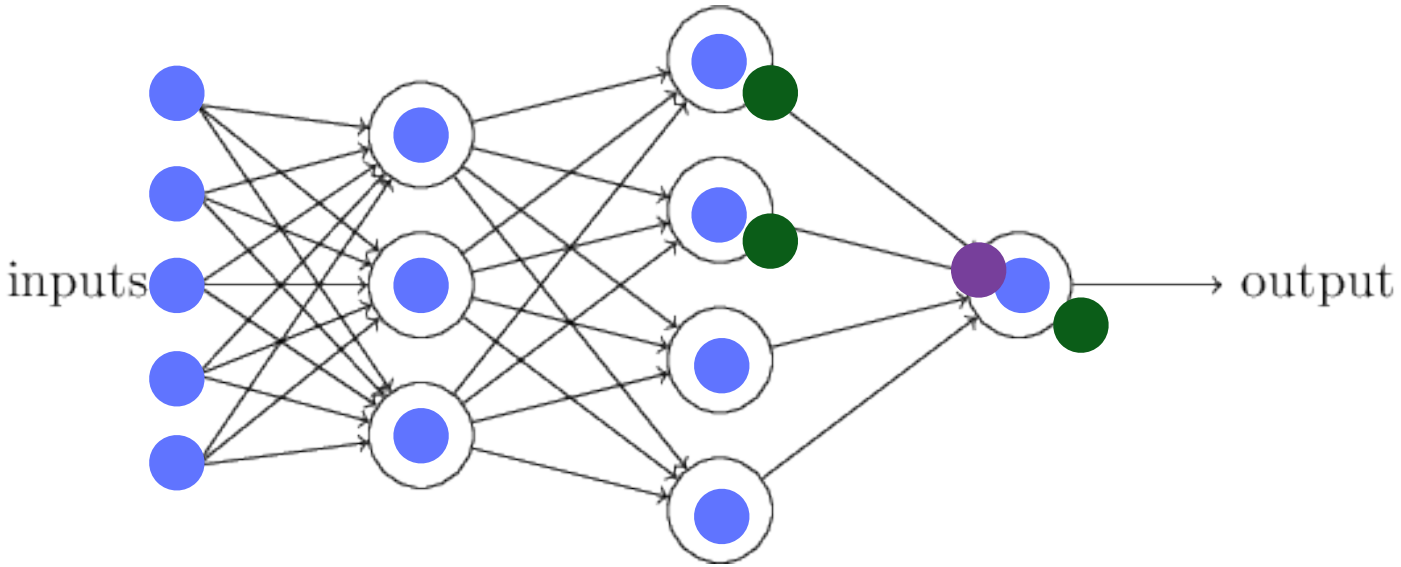
Backpropagation:
Step 2 - Error backpropagation



We then operate on the hidden layer.
The new delta of each hidden neuron is

error● := Sum(next neuron weight * next neuron delta)
delta● := error * transferDerivative(output)

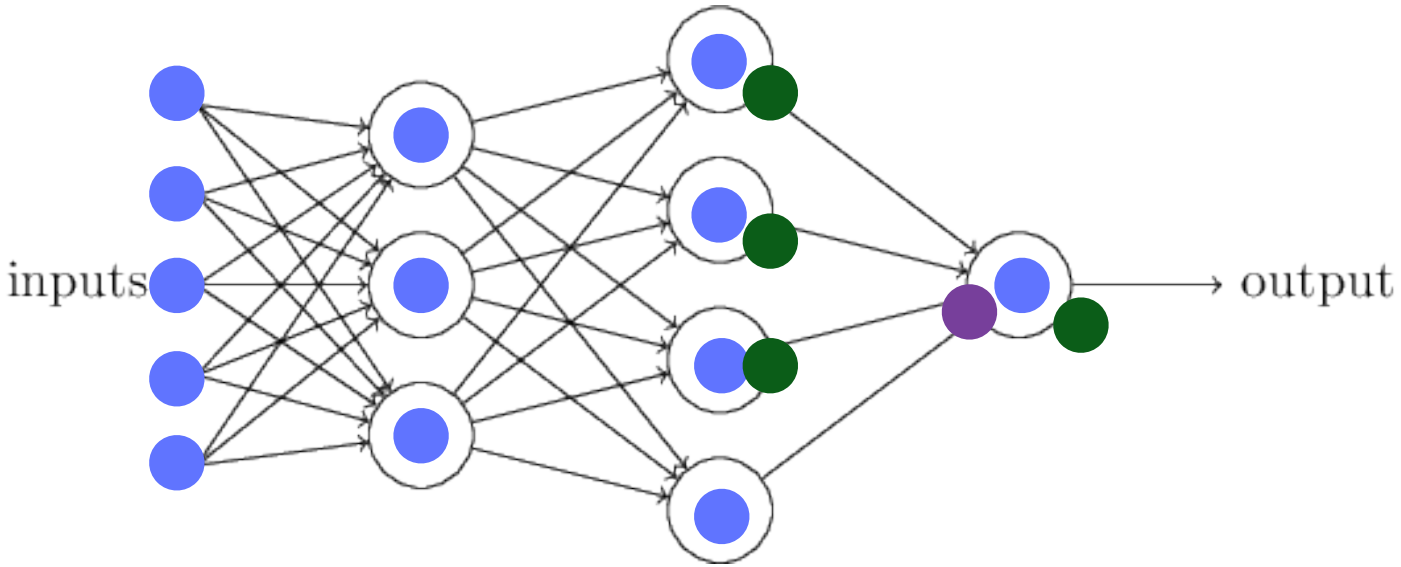
Backpropagation:
Step 2 - Error backpropagation



We then operate on the hidden layer.
The new delta of each hidden neuron is

error ● := Sum(next neuron weight * next neuron delta)
delta ● := error * transferDerivative(output)

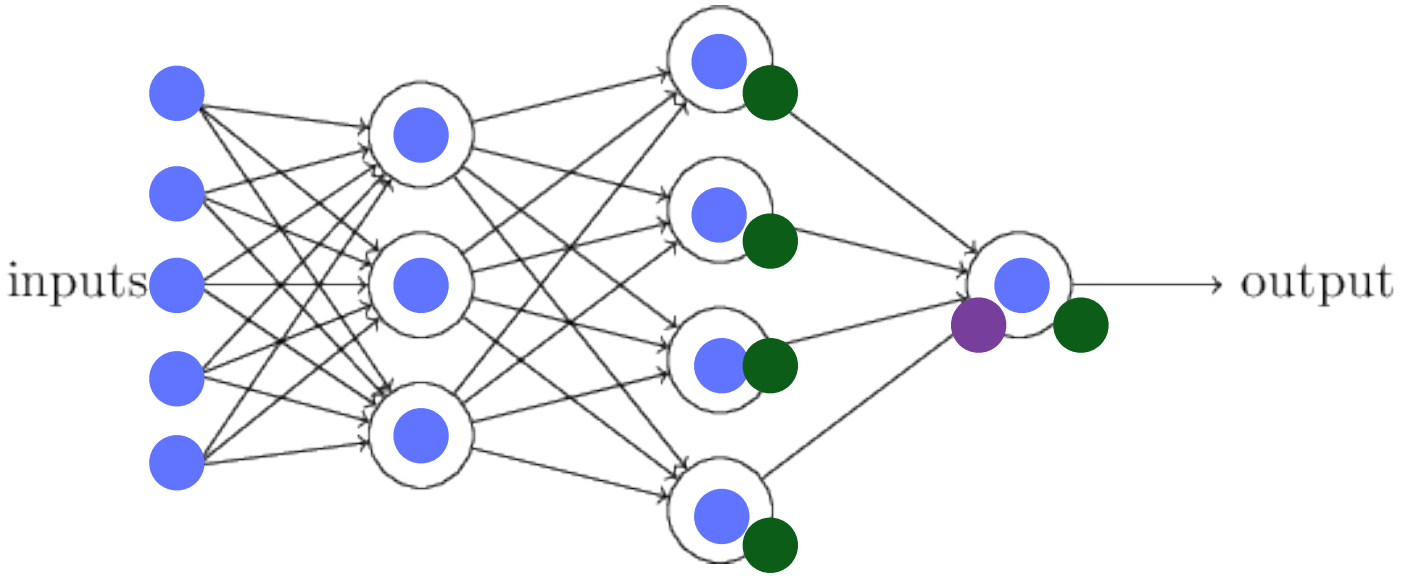
Backpropagation:
Step 2 - Error backpropagation



We then operate on the hidden layer.
The new delta of each hidden neuron is

error● := Sum(next neuron weight * next neuron delta)
delta● := error * transferDerivative(output)

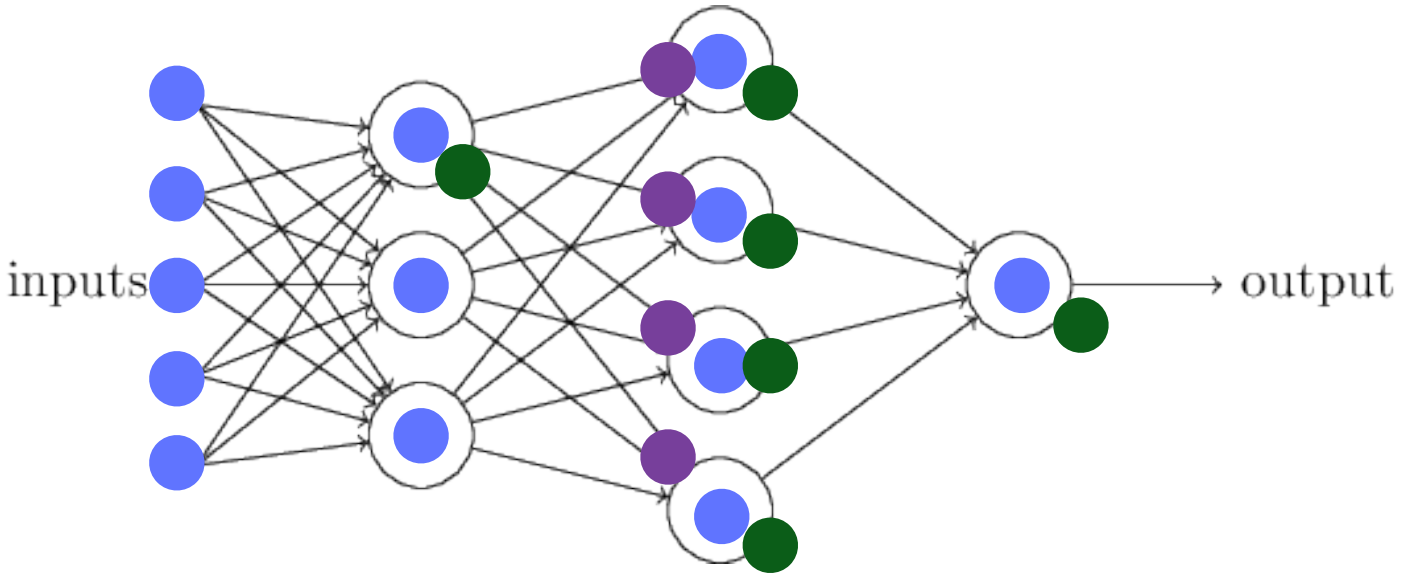
Backpropagation:
Step 2 - Error backpropagation



We then operate on the hidden layer.
The new delta of each hidden neuron is

error ● := Sum(next neuron weight * next neuron delta)
delta ● := error * transferDerivative(output)

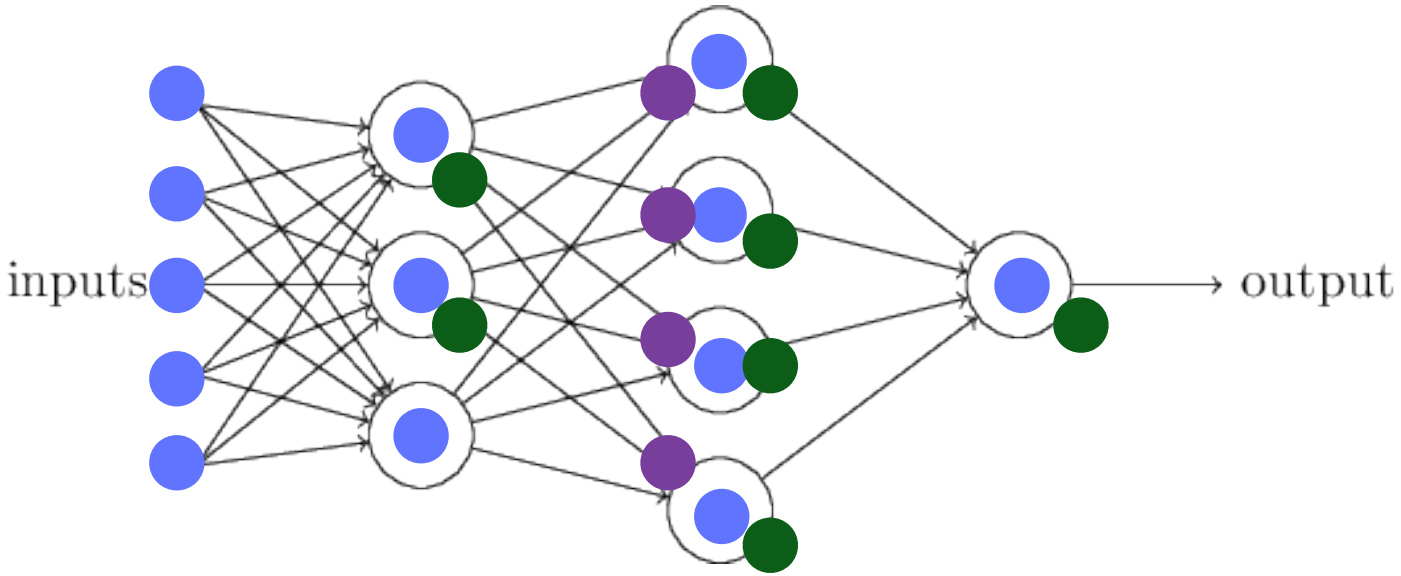
Backpropagation:
Step 2 - Error backpropagation



We then operate on the hidden layer.
The new delta of each hidden neuron is

error● := Sum(next neuron weight * next neuron delta)
delta● := error * transferDerivative(output)

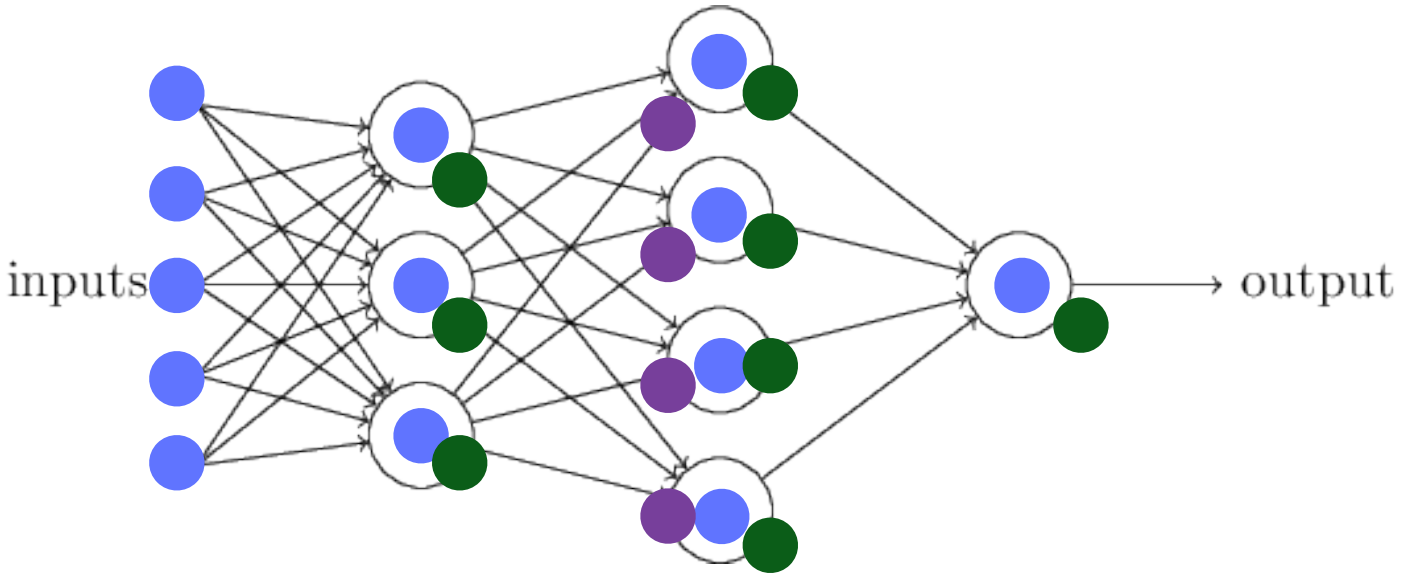
Backpropagation:
Step 2 - Error backpropagation



We then operate on the hidden layer.
The new delta of each hidden neuron is

error ● := Sum(next neuron weight * next neuron delta)
delta ● := error * transferDerivative(output)

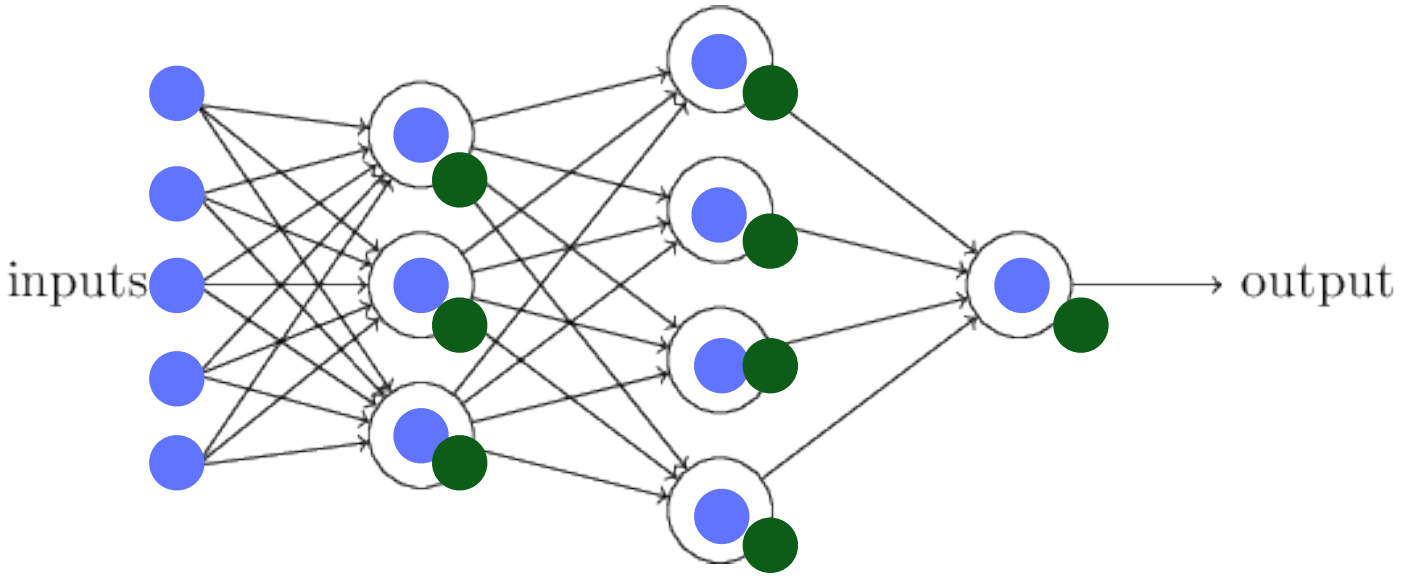
Backpropagation:
Step 2 - Error backpropagation



We then operate on the hidden layer.
The new delta of each hidden neuron is

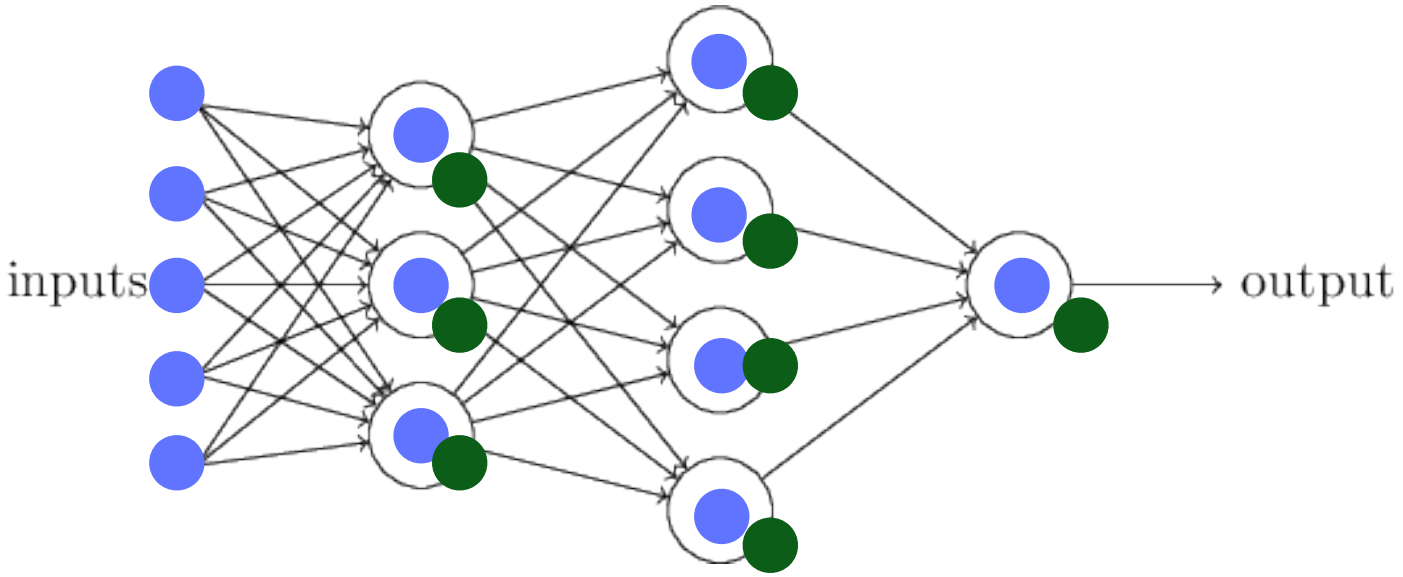
error ● := Sum(next neuron weight * next neuron delta)
delta ● := error * transferDerivative(output)

Backpropagation:
Step 2 - Error backpropagation



All the delta have been propagated.
We can now enter the third phase, with the weight and bias adjustments

Backpropagation:
Step 3 - Updating weights and bias

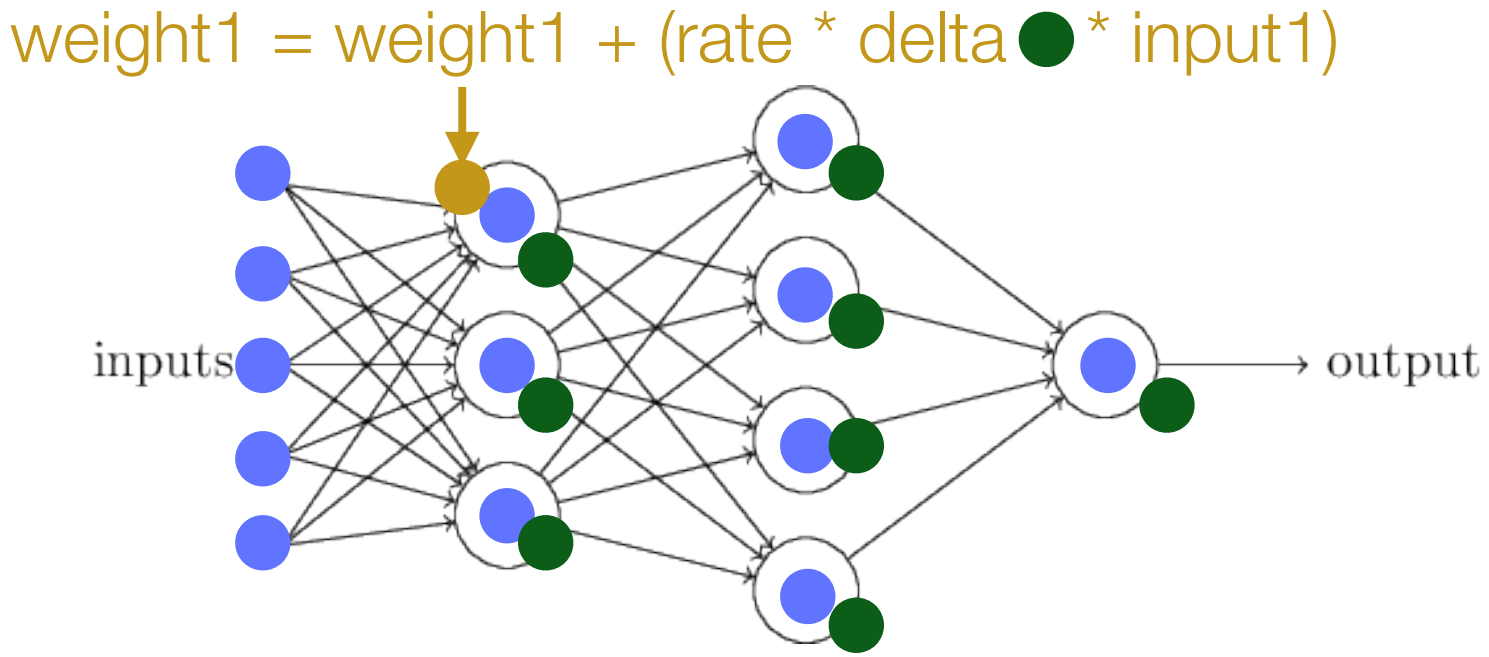


We compute a new weight for each neuron

weightN = weightN + (learningRate * delta * inputN)

bias = bias + (learningRate * delta)

Backpropagation:
Step 3 - Updating weights and bias

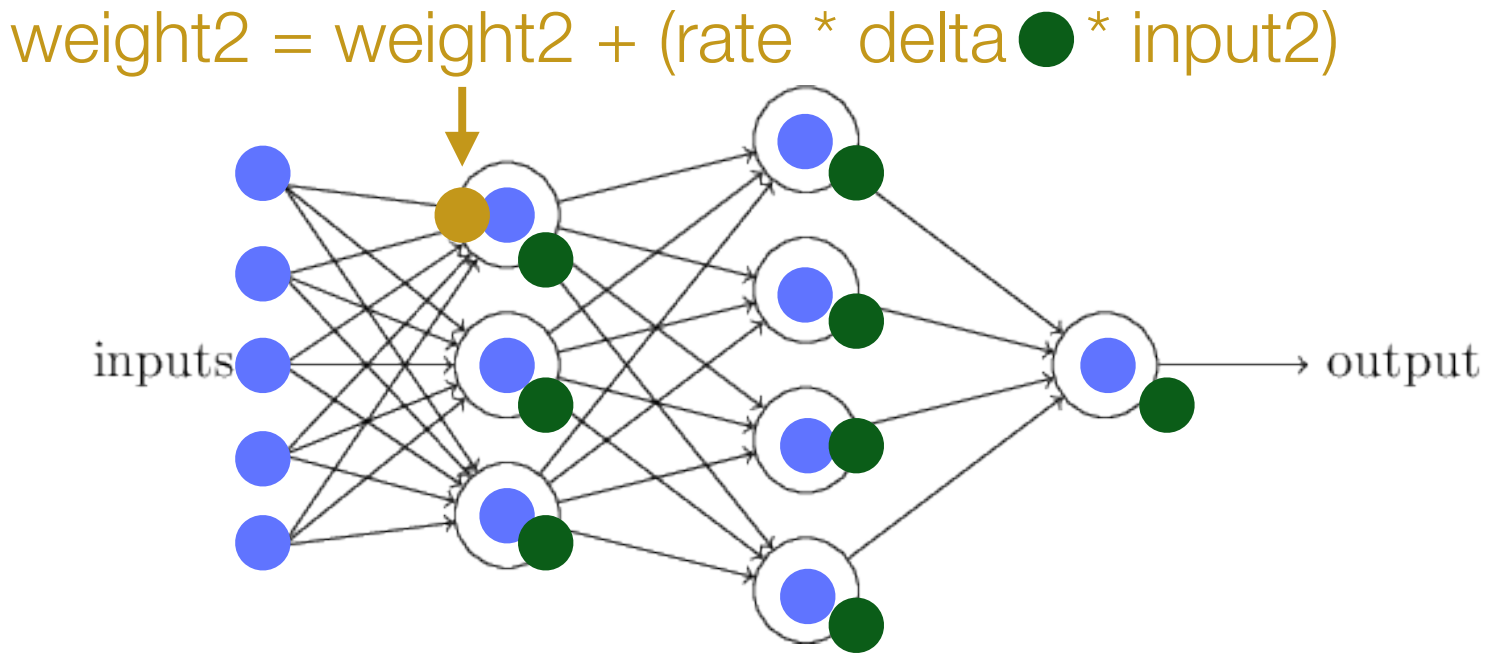


We compute a new weight for each neuron

weightN = weightN + (learningRate * delta * inputN)

bias = bias + (learningRate * delta)

Backpropagation:
Step 3 - Updating weights and bias

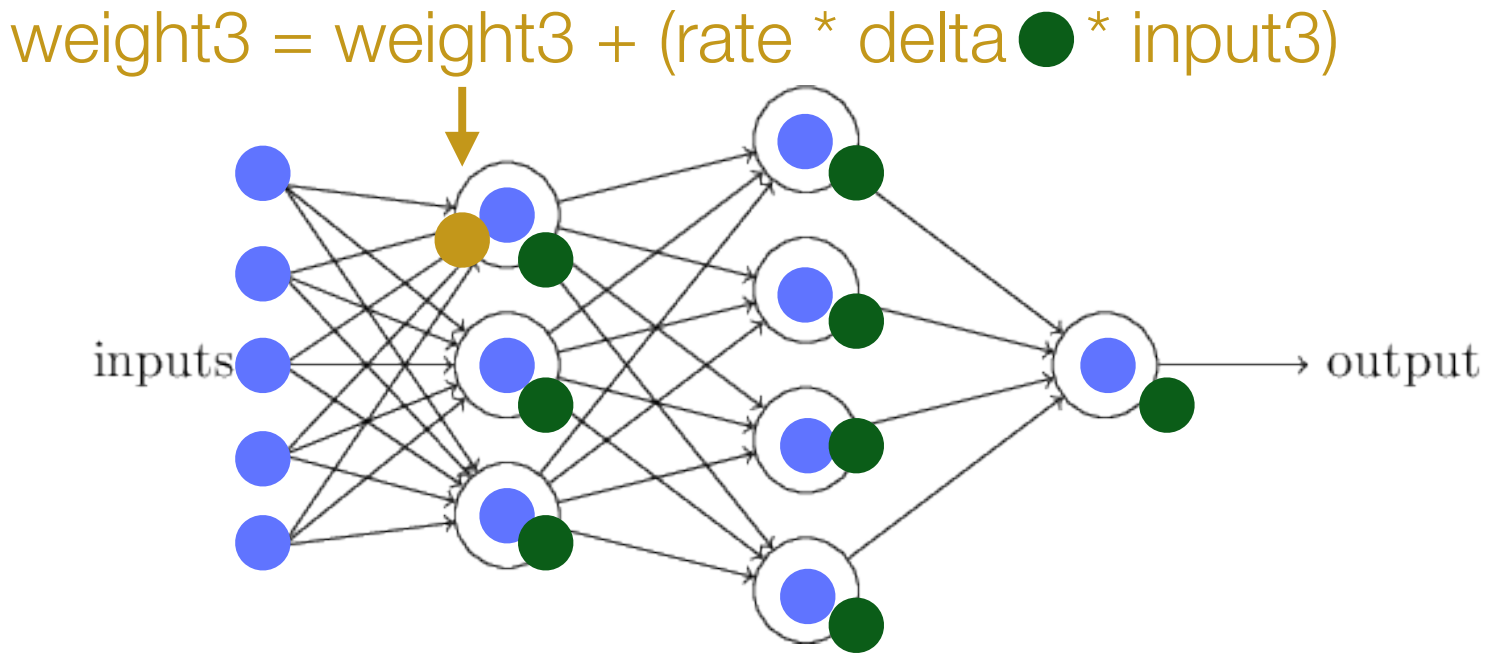


We compute a new weight for each neuron

weightN = weightN + (learningRate * delta * inputN)

bias = bias + (learningRate * delta)

Backpropagation:
Step 3 - Updating weights and bias

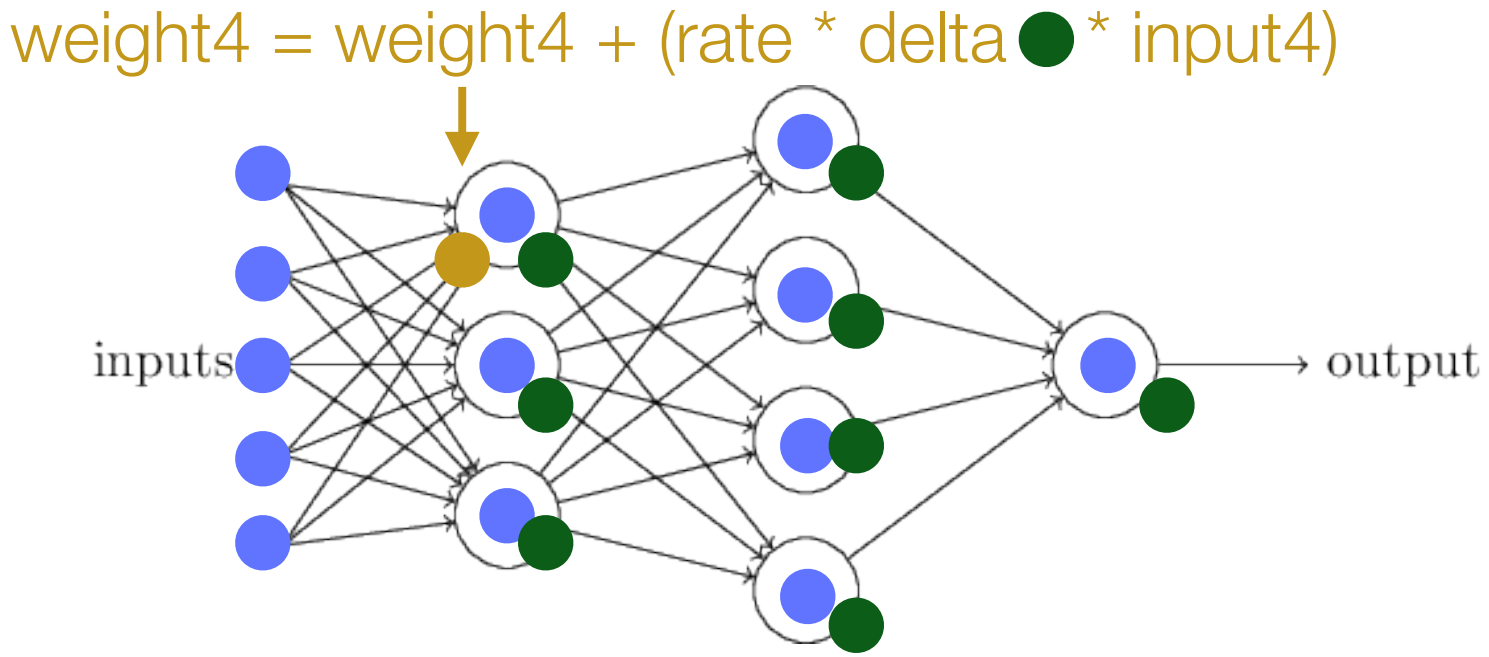


We compute a new weight for each neuron

weightN = weightN + (learningRate * delta * inputN)

bias = bias + (learningRate * delta)

Backpropagation:
Step 3 - Updating weights and bias



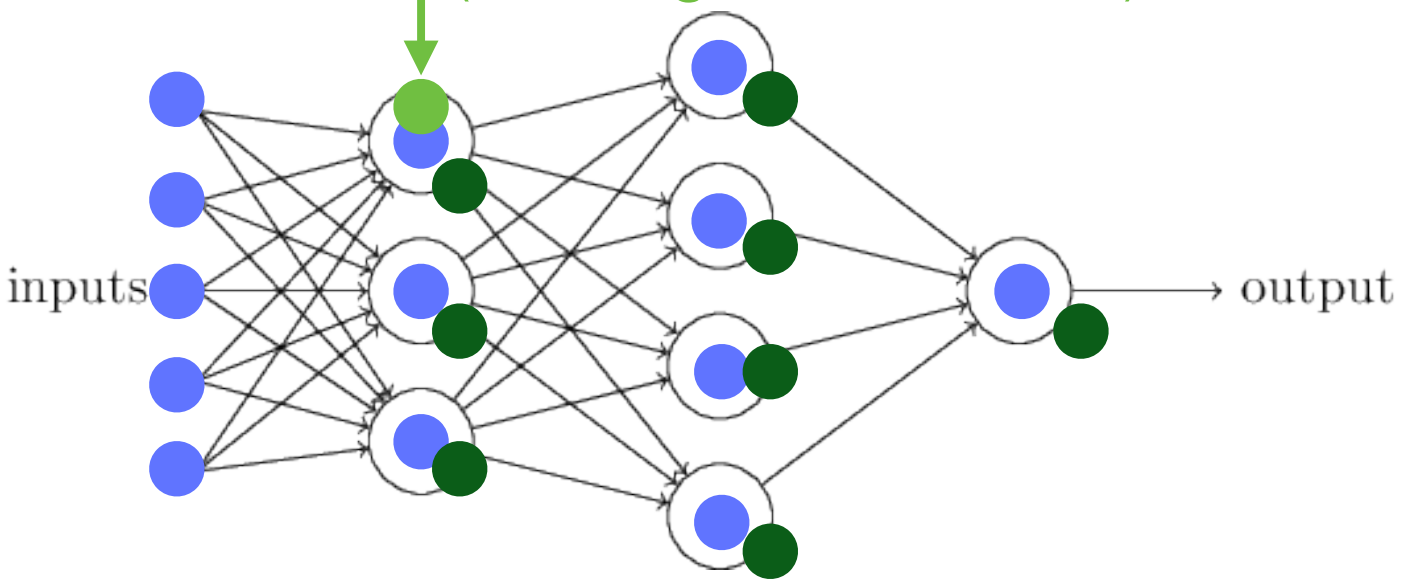
We compute a new weight for each neuron

weightN = weightN + (learningRate * delta * inputN)

bias = bias + (learningRate * delta)

Backpropagation:
Step 3 - Updating weights and bias

bias = bias + (learningRate * delta●)

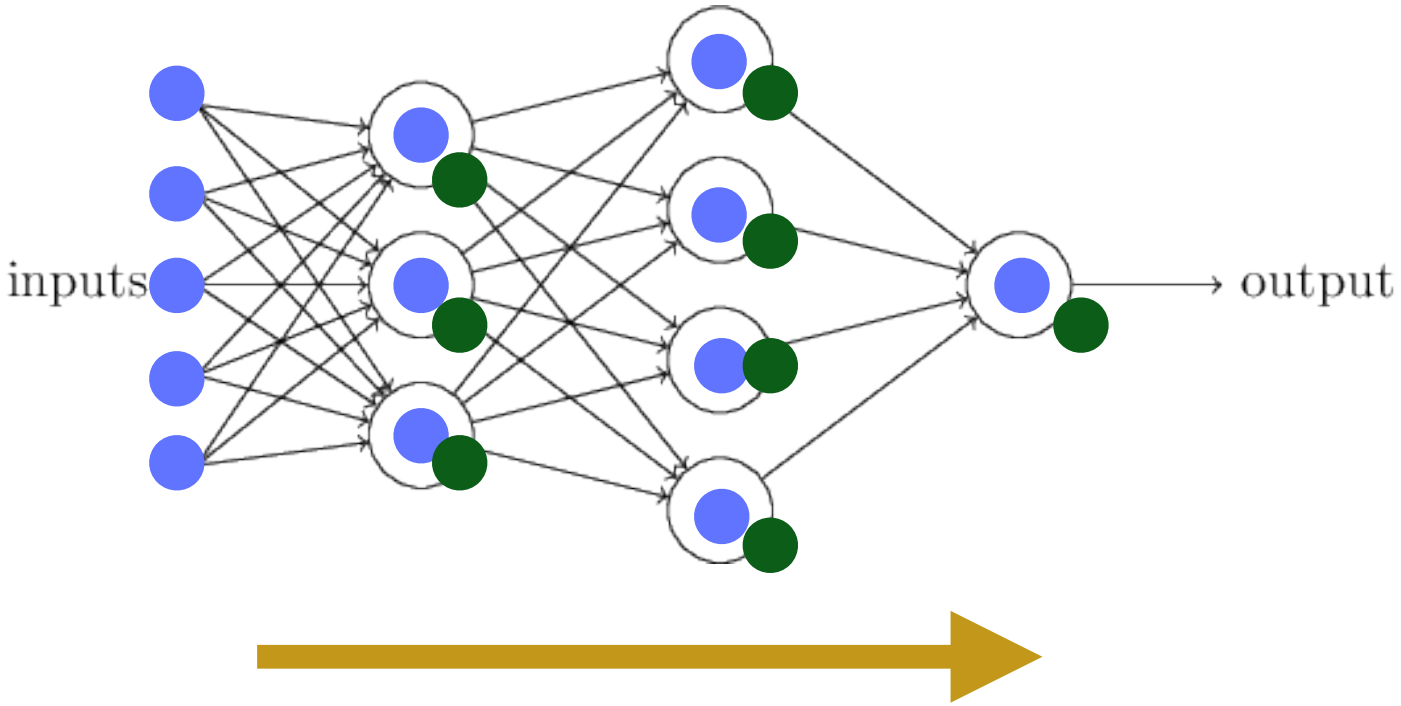


We compute a new weight for each neuron

weightN = weightN + (learningRate * delta * inputN)

bias● = bias + (learningRate * delta)

Backpropagation:
Step 3 - Updating weights and bias



We update the weights for all the neurons, in all the layers.
Remember: each input of a neuron is an output in the previous layer

Important aspects

In total, learning implies 3 different phases:

First: from left to right, to compute the output

Second: from right to left, to compute the error

Third: over all the neurons to update the weights and the bias

For the error back propagation, there are two cases: being in the *output layer*, or in an *hidden layer*

Backpropagation: Pharo code

```
NeuralNetwork>>train: inputs desiredOutput: expectedOutputs
  "Train the network with a set of inputs, and a set of expected output.
  This method return the error"
  | outputs |
  outputs := self feed: inputs.
  self backwardPropagateError: expectedOutputs.
  self updateWeight: inputs.
```

Backpropagation: Pharo code

This code begins the method named “train:desiredOutput:” in the class NeuralNetwork

```
NeuralNetwork>>train: inputs desiredOutput: expectedOutputs
    "Train the network with a set of inputs, and a set of expected output.
    This method return the error"
    | outputs |
    outputs := self feed: inputs.
    self backwardPropagateError: expectedOutputs.
    self updateWeight: inputs.
```

Backpropagation: Pharo code

The method requires two arguments, named inputs and expectedOutputs

```
NeuralNetwork>>train: inputs desiredOutput: expectedOutputs
    "Train the network with a set of inputs, and a set of expected output.
    This method return the error"
    | outputs |
    outputs := self feed: inputs.
    self backwardPropagateError: expectedOutputs.
    self updateWeight: inputs.
```

Backpropagation: Pharo code

```
NeuralNetwork>>train: inputs desiredOutput: expectedOutputs
    "Train the network with a set of inputs, and a set of expected
output.
    This method re
    | outputs |
    outputs := sel
    self backwardPropagateError: expectedOutputs.
    self updateWeight: inputs.
```

Temporary variable definition

Backpropagation: Pharo code

```
  Neural                                                    puts
    "Tra                                                     expected
output.
    This
    | outputs
    outputs := self feed: inputs.
    self backwardPropagateError: expectedOutputs.
    self updateWeight: inputs.
```

The message (feed:, inputs) is sent to the object self
In Java, it would be written: this.feed(inputs)

Backpropagation

```
NeuralNetwork>>backwardPropagateError: expected
      self lastLayer backwardPropagateError: expected.
```

Backpropagation

```
NeuronLayer>>backwardPropagateError: expected
    "This is a recursive method. The back propagation begins
with the output layer (i.e., the last later)"

    "We are in the output layer"
    neurons with: expected do: [ :neuron :exp |
        | theError |
        theError := exp - neuron output.
        neuron adjustDeltaWith: theError ].

    self previousLayer notNil
        ifTrue: [
            self previousLayer backwardPropagateError ].
```

Backpropagation

```
NeuronLayer>>backwardPropagateError
    "This is a recursive method. The back propagation begins with the output
layer (i.e., the last later)"
    "We are in an hidden layer”

    neurons doWithIndex: [ :neuron :j |
        | theError |
        theError := 0.0.
        self nextLayer neurons do: [ :nextNeuron |
            theError:=theError + ((nextNeuron weights at: j) *nextNeuron delta) ].
        neuron adjustDeltaWith: theError
    ].
    self previousLayer notNil
        ifTrue: [
             self previousLayer backwardPropagateError ].
```

Backpropagation

```
NeuronLayer>>transferDerivative: output
 "Calculate the derivative of a neuron output"
 ^ output * (1.0 - output)
```

Backpropagation

```
NeuralNetwork>>updateWeight: initialInputs
   "Update the weights of the neurons using the initial inputs"
   self firstHiddenLayer updateWeight: initialInputs
```

Backpropagation

```
NeuralNetwork>>updateWeight: initialInputs
    "Update the weights of the neuron based on the set of initial input. This method assumes
that the receiver of the message invoking that method is the first hidden layer."
    | inputs |
    learningRate := 0.5.
    self assert: [ neurons allSatisfy: [ :n | n delta notNil ] ].

    self previousLayer
        ifNil: [ inputs := initialInputs ]
        ifNotNil: [ inputs := self previousLayer neurons collect: #output ].

    neurons do: [ :n |
        n adjustWeightWithInput: inputs learningRate: learningRate.
        n adjustBiasUsingLearningRate: learningRate ].

    self nextLayer ifNotNil: [
        self nextLayer updateWeight: initialInputs ]
```

Backpropagation

```
Neuron>>adjustBiasUsingLearningRate: learningRate
  bias := bias + (learningRate * delta)

Neuron>>adjustDeltaWith: anError
  delta := (anError * (self transferDerivative: output))

Neuron>>adjustWeightWithInput: inputs learningRate: learningRate
  inputs withIndexDo: [ :anInput :index |
      weights at: index put: ((weights at: index) + (learningRate * delta *
anInput)) ].

Neuron>>transferDerivative: output
  "Calculate the derivative of a neuron output"
  ^ output * (1.0 - output)
```

Classifying some data

Iris



1. Title: Iris Plants Database
   Updated Sept 21 by C.Blake - Added discrepency information

2. Sources:
   (a) Creator: R.A. Fisher
   (b) Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
   (c) Date: July, 1988

3. Past Usage:
   - Publications: too many to mention!!! Here are a few.
   1. Fisher,R.A. "The use of multiple measurements in taxonomic problems" Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to Mathematical Statistics" (John Wiley, NY, 1950).
   2. Duda,R.O., & Hart,P.E. (1973) Pattern Classification and Scene Analysis. (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.
   3. Dasarathy, B.V. (1980) "Nosing Around the Neighborhood: A New System Structure and Classification Rule for Recognition in Partially Exposed Environments". IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. PAMI-2, No. 1, 67-71.
      -- Results:
         -- very low misclassification rates (0% for the setosa class)
   4. Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE Transactions on Information Theory, May 1972, 431-433.
      -- Results:
         -- very low misclassification rates again
   5. See also: 1988 MLC Proceedings, 54-64. Cheeseman et al's AUTOCLASS II conceptual clustering system finds 3 classes in the data.

4. Relevant Information:
   --- This is perhaps the best known database to be found in the pattern recognition literature. Fisher's paper is a classic in the field and is referenced frequently to this day. (See Duda & Hart, for example.) The data set contains 3 classes of 50 instances each, where each class refers to a type of iris plant. One class is linearly separable from the other 2; the latter are NOT linearly separable from each other.
   --- Predicted attribute: class of iris plant.

Iris dataset

Obtained from:

https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data

Description of the values:

https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.names

```
g := RTGrapher new.

d := RTData new.
d dotShape
    if: [ :row | row last = 'Iris-setosa' ] fillColor: Color blue;
    if: [ :row | row last = 'Iris-versicolor' ] fillColor: Color white;
    if: [ :row | row last = 'Iris-virginica' ] fillColor: Color green.

d points: NNDataset new irisDataset.
d x: #first.
d y: #third.
g add: d.

g
```

```
| b data block d lb |
data := RTTabTable new input: (TRPlatform current downloadContent: 'http://mbostock.github.io/d3/
talk/20111116/iris.csv') usingDelimiter: $,.
data
    removeFirstRow;
    convertColumns: #( 2 3 4 5) to: #asNumber.

b := RTScatterplotMatrix new.

b objects: data values.

block := [ :n | n == n asInteger
    ifTrue: [ n asInteger ]
    ifFalse: [ n asFloat ] ].
b axisX
    numberOfTicks: 5;
    rotateLabels;
    labelConversion: block.
b axisY
    numberOfTicks: 5;
    labelConversion: block.

b lineShape: (RTStyledMultiLine new
    dashedLine;
    width: 0.5; yourself
    ).

d := Dictionary new
    at: 'setosa' put: Color red;
    at: 'versicolor' put: Color green;
    at: 'virginica' put: Color blue; yourself.
b shape circle
    size: 3.5;
    color: [ :a | d at: a first ].
```

```
dataSet := NNDataset new irisDatasetWithNumericalLast.
dataSet := dataSet shuffled.
cut := (dataSet size * 0.5) asInteger.
dataSetTraining := dataSet first: cut.
dataSetGuessing := dataSet last: (dataSet size - cut).


p := NeuralNetwork new.
p configure: 4 numberOfHidden: 5 nbOfOutput: 3.

p train: dataSetTraining nbEpoch: 2000.


precision := (dataSetGuessing select: [ :row |
    (p predict: row allButLast) = row last ]) size / dataSetGuessing size.

'Prediction of precision = ', (((precision round: 4) * 100) asString), '%'
```

Prediction of precision = 96.0%

Genetic Algorithm

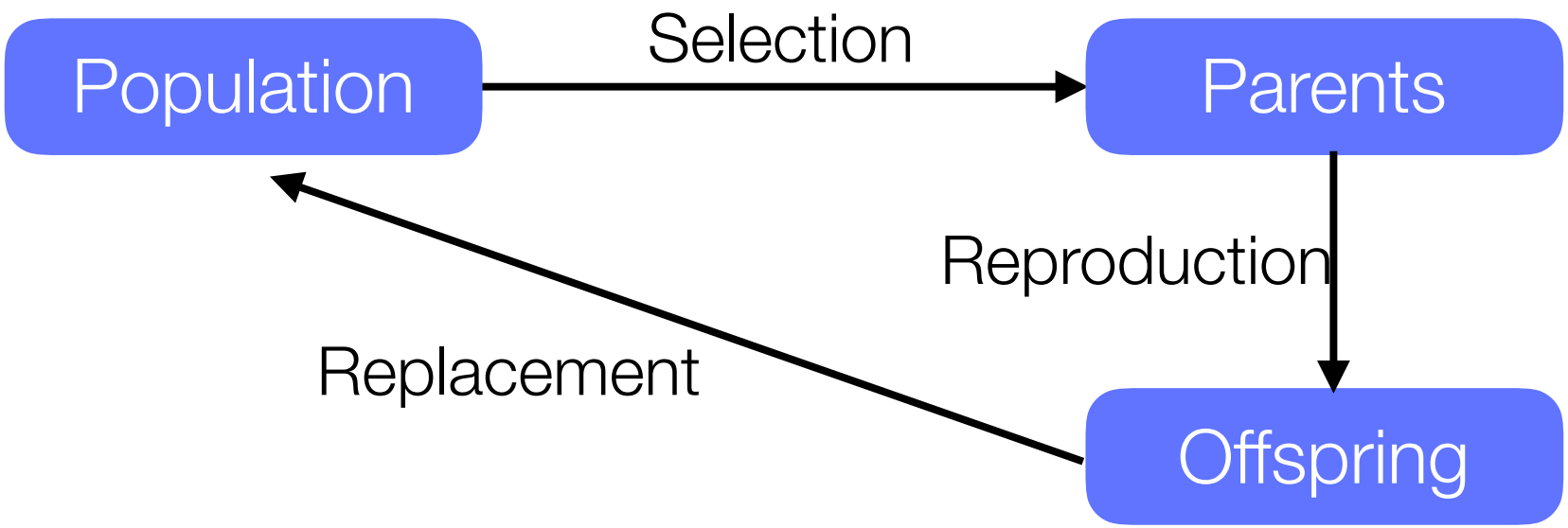
“The fact that life evolved out of nearly nothing, some 10 billion years after the universe evolved out of literally nothing, is a fact so staggering that I would be mad to attempt words to do it justice.”

— Richard Dawkins

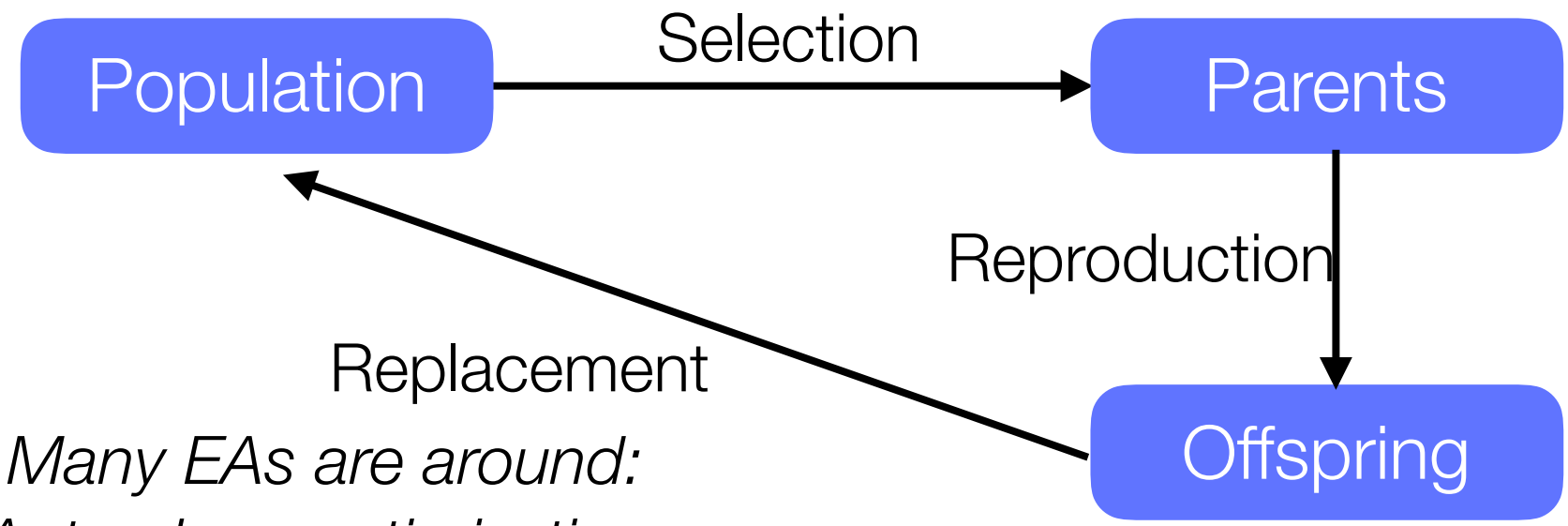
“One general low, leading to the advancement of all organic beings, namely, vary, let the strongest live and the weakest die”

— Charles Darwin

Flow - chart of an evolution algorithm



Flow - chart of an evolution algorithm



Many EAs are around:
Ant colony optimization,
Artificial immune system,
Cultural algorithms,
Genetic Algorithm,
Genetic Programming,
Grammatical evolution,
…

Evolution algorithm

May be written as:

$$x[t + 1] = v(s(x[t]))$$

where:

x[t] is the population at time t under a representation x

s is the selection operator

v is a random variation operator

Evolution algorithm

Evolution algorithms have many advantages, including:

Offer a framework such that it is comparably easy to incorporate prior knowledge about the problem.

May be combined with other optimization techniques. For example, could be used to turning weights in a neural networks

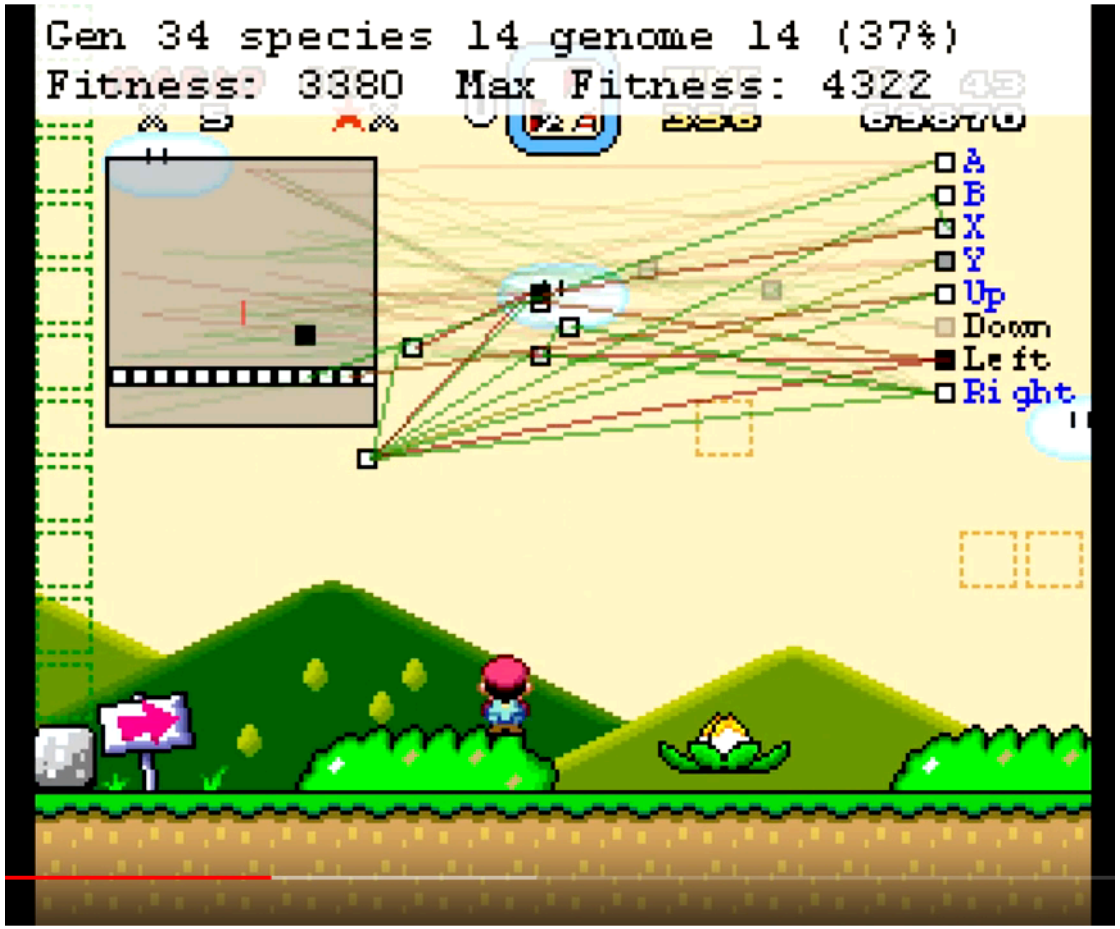
Genetic Algorithm in a Nutshell

Evolutionary computation technique that automatically solves problems without specifying the form or structure of the solution in advance

Generally speaking, *genetic algorithms are simulations of evolution*, of what kind ever

Combining NN and GA

https://www.youtube.com/watch?v=qv6UVOQ0F44



That’s all Folk!

You have built a real neural networks

You can use it for many things:

- Data classification
- Word Embedding
- Image recognition (the MNIST benchmark works well)

That's all Folk!

There are plenty of opportunities to apply AI techniques in the field of software system constructions

Software testing, Software automatic configuration,

Software product lines, robotic, …

A practical approach to Neural Networks

Alexandre Bergel
University of Chile
http://bergel.eu